如果上面的最优基解 x^* 不是整数解，那就一定存在一行 u 在最优基单纯形表中的右端常数项 $\bar{a}_{u0} \notin \mathbb{Z}^1$（即 $\bar{a}_{u0}$ 是小数）。下面任选一个这样的行，该行的 Gomory 割可以表示为

$$x_{B_u} + \sum_{j \in \mathrm{NB}} \lfloor \bar{a}_{uj} \rfloor \cdot x_j \leqslant \lfloor \bar{a}_{u0} \rfloor \tag{11.26}$$

把式 (11.24) 和式 (11.26) 相减，消除 x_{B_u} 即可得到下面的式子：

$$\sum_{j \in \mathrm{NB}} (\bar{a}_{uj} - \lfloor \bar{a}_{uj} \rfloor) \cdot x_j \geqslant \bar{a}_{u0} - \lfloor \bar{a}_{u0} \rfloor \tag{11.27}$$

令

$$f_{uj} = \bar{a}_{uj} - \lfloor \bar{a}_{uj} \rfloor$$
$$f_{u0} = \bar{a}_{u0} - \lfloor \bar{a}_{u0} \rfloor$$

则表达式 (11.27) 可以表示成

$$\sum_{j \in \mathrm{NB}} f_{uj} x_j \geqslant f_{u0} \tag{11.28}$$

根据上述的描述可知，对于任意一行，比如第 u 行，均有 $0 \leqslant f_{uj} < 1$，且 $0 < f_{u0} < 1$。并且在最优解中，所有非基变量 $x_j^* = 0, \forall j \in \mathrm{NB}$，因此表达式 (11.28) 的左端项 $\sum\limits_{j \in \mathrm{NB}} f_{uj} x_j = \sum\limits_{j \in \mathrm{NB}} f_{uj} \cdot 0 = 0$。因此这个割可以割掉小数最优解 x^*。原因是如果 x^* 是小数，且因为 $x_j^* = 0, \forall j \in \mathrm{NB}$，根据上面的讨论，约束表达式 (11.27) 中的左边 $= 0$，右边 $f_{u0} = \bar{a}_{u0} - \lfloor \bar{a}_{u0} \rfloor > 0$，约束表达式 (11.27) 不成立。因此 x^* 这个小数解不符合这个割约束，所以小数解 x^* 就被割平面割掉了，不可能出现在最优解中。

另外，可以看到式 (11.26) 中的左端项和右端项的差，也就是式 (11.28) 的左端项和右端项的差，即

$$s = -f_{u0} + \sum_{j \in \mathrm{NB}} f_{uj} x_j \tag{11.29}$$

实际上 s 就是这一条约束的松弛变量。其类型同样也是一个非负的整数变量（如果整数规划的右端常数和约束系数都是整数，整数和整数的和、差都是整数，那么松弛变量 s 也必须是非负的整数变量）。

下面用一个例子来更具体了解 Gomory 割平面算法。

$$\max \quad z = 4x_1 - x_2 \tag{11.30}$$
$$\text{s.t.} \quad 7x_1 - 2x_2 \leqslant 14 \tag{11.31}$$
$$x_2 \leqslant 3 \tag{11.32}$$
$$2x_1 - 2x_2 \leqslant 3 \tag{11.33}$$
$$x_1, x_2 \geqslant 0 \text{ 且 } \in \mathbb{Z} \tag{11.34}$$

添加松弛变量 x_3, x_4, x_5，这里松弛变量也必须是非负的整数变量，因为该整数规划的约束系数和右端项都是整数，因此松弛变量也必须是非负的整数。求解这个整数规划，得到下面的最优单纯形表：

$$\begin{aligned}
\max\ z=&\frac{59}{7} && -\frac{4}{7}x_3-\frac{4}{7}x_4 && \\
\text{s.t.}\quad & x_1 && +\frac{1}{7}x_3+\frac{2}{7}x_4 && =\frac{20}{7}\\
& x_2 && +x_4 && =3\\
& && -\frac{2}{7}x_3+\frac{10}{7}x_4+x_5 && =\frac{23}{7}\\
& x_1,\ x_2,\ x_3,\ && x_4,\ && x_5\geqslant 0\quad \text{且}\in\mathbb{Z}
\end{aligned}\tag{11.35}$$

该线性松弛问题的最优解为 $\boldsymbol{x}=(20/7,3,0,0,23/7)\notin\mathbb{Z}_+^5$，选取第一行，第一行对应的基变量 x_1 的取值是分数，因此可以生成一条割

$$\frac{1}{7}x_3+\frac{2}{7}x_4\geqslant\frac{6}{7}$$

其中 6/7 是右端常数 20/7 的小数部分。即松弛变量为

$$s=-\frac{6}{7}+\frac{1}{7}x_3+\frac{2}{7}x_4$$

且 $s,x_3,x_4\geqslant 0$ 为整数。将这条约束加进去，继续用单纯形法求解添加割平面后的模型，得到下面的最优单纯形表：

$$\begin{aligned}
\max\ z=&\frac{15}{2} && -\frac{1}{2}x_5-3s && \\
& x_1 && +s && =\frac{20}{7}\\
& x_2 && -\frac{1}{2}x_5+s && =\frac{1}{2}\\
& x_3 && -x_5\ -5s && =1\\
& && x_4+\frac{1}{2}x_5+6s && =\frac{5}{2}\\
& x_1,\ x_2,\ x_3,\ && x_4,\ x_5,\ && s\ \geqslant 0\quad \text{且}\in\mathbb{Z}
\end{aligned}$$

现在最优解为 $\boldsymbol{x}=(2,1/2,1,5/2,0)\notin\mathbb{Z}_+^5$。取小数变量 x_2 构造 Gomory 割平面，得

$$\frac{1}{2}x_5\geqslant\frac{1}{2}$$

或者说，得到松弛变量

$$t=-\frac{1}{2}x_5+\frac{1}{2}$$

且 $t\geqslant 0$ 为整数。继续求解得

$$\begin{aligned}
\max\ z=7\quad & && -3s-t && \\
& x_1 && +s && =2\\
& x_2 && +s\ -t && =1\\
& x_3 && -5s-2t && =2
\end{aligned}$$

$$
\begin{aligned}
x_4 \quad & +6s+t \ = 2 \\
x_5 \quad & -t \ = 1 \\
x_1,\ x_2,\ x_3,\ x_4,\ x_5,\ s,\ t \ & \geqslant 0 \text{ 且 } \in \mathbb{Z}
\end{aligned}
$$

此时，模型得到了整数解 $(x_1, x_2) = (2, 1)$。

继续上面的讨论，下面为第一个割平面：

$$\frac{1}{7}x_3 + \frac{2}{7}x_4 \geqslant \frac{6}{7}$$

从最优单纯形表 (11.35) 中，得知 $x_2 + x_4 = 3 \rightarrow x_4 = 3 - x_2$，且 $x_1 + \frac{1}{7}x_3 + \frac{2}{7}x_4 = \frac{20}{7}$，因此 $7x_1 + x_3 + 2x_4 = 20$，所以 $x_3 = 20 - 7x_1 - 2x_4 = 20 - 7x_1 - 2(3 - x_2) = 14 - 7x_1 + 2x_2$。因此可以得到

$$\frac{1}{7}(14 - 7x_1 + 2x_2) + \frac{2}{7}(3 - x_2) \geqslant \frac{6}{7}$$

化简得

$$x_1 \leqslant 2$$

读者可以自行画出该问题的可行域进行验证。$x_1 \leqslant 2$ 确实是一个有效不等式，并且确实将第一次迭代产生的小数最优解 $(x_1, x_2) = (20/7, 3)$ 给切掉了。同理，在第二次迭代中，加入下面的割平面：

$$\frac{1}{2}x_5 \geqslant \frac{1}{2}$$

也可以通过替换，合并化简得

$$x_1 - x_2 \leqslant 1$$

同样，也是将对应的小数解 $(x_1, x_2) = (2, 1/2)$ 给割掉了。

但是，每次都这样去替换非常麻烦，如果有一个通用的规则，能够直接找到这样一个割平面的表达式，并且这个割平面中涉及的变量全部都是最原始的决策变量，而不涉及后来加入的松弛变量，那就非常方便了。下面的定理就给出了找出 Gomory 割平面的一般方式。

定理 11.3.1 （Wolsey，1998）*如果 $\boldsymbol{B}$ 是线性松弛问题的最优基的约束矩阵，$\boldsymbol{B}^{-1}$ 是 $\boldsymbol{B}$ 的逆矩阵，$\boldsymbol{\lambda}$ 是 $\boldsymbol{B}^{-1}$ 的第 u 行，且 $q_i = \lambda_i - \lfloor \lambda_i \rfloor, \forall i = 1, 2, \cdots, m$，则 Gomory 割平面 $\sum\limits_{j \in \mathrm{NB}} f_{uj}x_j \geqslant f_{u0}$ 用原始变量可以表示为*

$$\sum_{j}^{n} \lfloor \boldsymbol{q}\boldsymbol{a}_j \rfloor x_j \leqslant \lfloor \boldsymbol{q}\boldsymbol{b} \rfloor \tag{11.36}$$

该不等式的英文名称为 Chvátal-Gomory inequality。这里 $\boldsymbol{q}$ 是 B^{-1} 的其中一行，是个行向量，而 $\boldsymbol{a}_j$ 是原始约束矩阵 $\boldsymbol{A}$ 中的第 j 列，是个列向量。$\boldsymbol{b}$ 是右端常数对应的列向量，其中 $\boldsymbol{q}\boldsymbol{a}_j, \boldsymbol{q}\boldsymbol{b}$ 都是向量相乘，这一点一定注意。

上例中的第一个割平面，见最优单纯形表 (11.35)，这里选择了第一行，而 $\boldsymbol{B}^{-1}$ 是由松弛变量 x_3, x_4, x_5 对应的列给出的，也就是

$$\boldsymbol{B}^{-1} = \begin{bmatrix} \frac{1}{7} & \frac{2}{7} & 0 \\ 0 & 1 & 0 \\ -\frac{2}{7} & \frac{10}{7} & 1 \end{bmatrix}$$

这个割平面对应第一行，也就是 $(1/7, 2/7, 0)$，因此 $\boldsymbol{q} = (1/7, 2/7, 0)$，而上面定理中 n 为原始变量的个数，该例子中 $n=2$，因此可以得到一个用原始的变量表示的 Gomory 割平面为

$$\sum_{j=1}^{n=2} \lfloor \boldsymbol{q}\boldsymbol{a}_j \rfloor x_j \leqslant \lfloor \boldsymbol{q}\boldsymbol{b} \rfloor$$

$$\Rightarrow \left\lfloor \begin{bmatrix} \frac{1}{7} & \frac{2}{7} & 0 \end{bmatrix} \begin{bmatrix} 7 \\ 0 \\ 2 \end{bmatrix} \right\rfloor \cdot x_1 + \left\lfloor \begin{bmatrix} \frac{1}{7} & \frac{2}{7} & 0 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \\ -2 \end{bmatrix} \right\rfloor \cdot x_2 \leqslant \left\lfloor \begin{bmatrix} \frac{1}{7} & \frac{2}{7} & 0 \end{bmatrix} \begin{bmatrix} 14 \\ 3 \\ 3 \end{bmatrix} \right\rfloor$$

$$\Rightarrow \left\lfloor \frac{11}{7} \right\rfloor x_1 + \lfloor 0 \rfloor \cdot x_2 \leqslant \left\lfloor \frac{20}{7} \right\rfloor$$

最终得到的割平面为

$$1 \cdot x_1 + 0 \cdot x_2 \leqslant 2$$

可以看到，这种方法得出的 Gomory 割平面和之前用变量替换化简的方式得出的割平面是一致的。

到这里，Gomory 割平面算法我们就介绍完毕了。

11.3.2 其他割平面算法

还有很多其他的割平面算法或者有效不等式，我们就不做详细介绍了，仅在这里列出名称，感兴趣的读者可以自行探索。

（1）BQP cut （Boolean Quadric Polytope cut）。
（2）Clique cut。
（3）Cover cut。
（4）Flow cover cut。
（5）Flow path cut。
（6）Gomory cut （Chvátal-Gomory cut）。
（7）GUB cover cut （Generalized Upper Bound cover cuts）。
（8）Implied bound cut。
（9）Infeasibility proof cut。
（10）MIP separation cut。
（11）MIR cut （Mixed Integer Rounding cut）。
（12）Mod-k cut。
（13）Network cut。
（14）Projected implied bound cut。
（15）Relax-and-lift cut。
（16）RLT cut （Reformulation Linearization Technique cut）。
（17）Strong-CG cut （Strong Chvátal-Gomory cut）。

（18）Sub-MIP cut。

（19）Zero-half cut。

上述 19 个 cut 也是 Gurobi 中几乎所有的 cut。到这里，割平面算法的部分就告一段落了。这些 cut 都是针对一般的 MIP 的 cut。对于特定问题，还有特定问题的 cut，比如 VRP 中的 robust cut 等（Costa et al., 2019），这些都需要根据问题的特性去设计。有时找到很有效的 cut 会对求解速度有非常大的提升。关于 VRP 的一些 cut，可以参考文献（Costa et al., 2019）。其中，介绍了鲁棒割平面（robust cuts），包括 k-路径割平面（k-path cuts）、子环路消除约束（subtour elimination constraints，SECs）和容量圆整割平面（rounded capacity cuts）；非鲁棒割平面（Non-robust cuts），包括子集行割平面（Subset Row Cuts，SRCs）、限制记忆的子集行割平面（Limited-memory SRCs）、基本割平面（Elementary cuts）、增强容量割平面（Strengthened capacity cuts）、团割平面（Clique cuts）、k 环消除割平面（k-cycle elimination cuts）、强度割平面（strong degree cuts）等。

接下来，我们就来探讨本章的重头戏，即分支切割算法，看看割平面算法如何与分支定界算法结合更有效地提升整数规划的求解效率。

11.4 分支切割算法：分支定界 + 割平面

11.4.1 分支切割算法伪代码

根据上述描述，可得分支切割（Branch and Cut）算法就是在执行分支定界算法时，在整个 BB tree 中的所有子节点都添加割平面的一种方法。虽然跟分支定界算法相比，似乎只有很小的不同，但是从整体思想来讲是有很大区别的。分支定界算法追求在每个节点尽快重新求解优化模型，努力去逼近凸包，从而更快地探索节点。而分支切割算法在每个节点处都争取做更多的事情，努力去割掉不可能存在整数可行解的区域，更快逼近凸包，去获得更好的 bound。因此，我们的目标不仅是利用割平面提升模型，让模型变得更紧，从而显著地减少 BB tree 中节点的个数，而且尝试一些其他的方法来帮助提升 bound。

在算法实际执行中，分支和割平面明显需要做一个权衡。如果在一个节点添加了过多的割平面，在求解节点的子问题时，就会花费很多时，并且存储整个 BB tree 的信息也是比较困难的。在分支定界算法中，在每个节点处，我们只增加了分支界限的约束。但是在分支切割算法中，我们就会使用一个叫作割平面池（Cut Pool）的集合来存储所有的割平面。另外，为了在节点处存储 bound 和比较好的可行基，我们需要说明哪些约束需要在哪些给定的点进行重新构建，为此我们需要在节点处存储指针信息，这些指针又指向割平面池中的哪些割平面。

为了让大家对分支切割算法的框架有一个更加系统的理解，我们参照文献（Wolsey, 1998），给出分支切割算法框图，如图 11.5 所示。

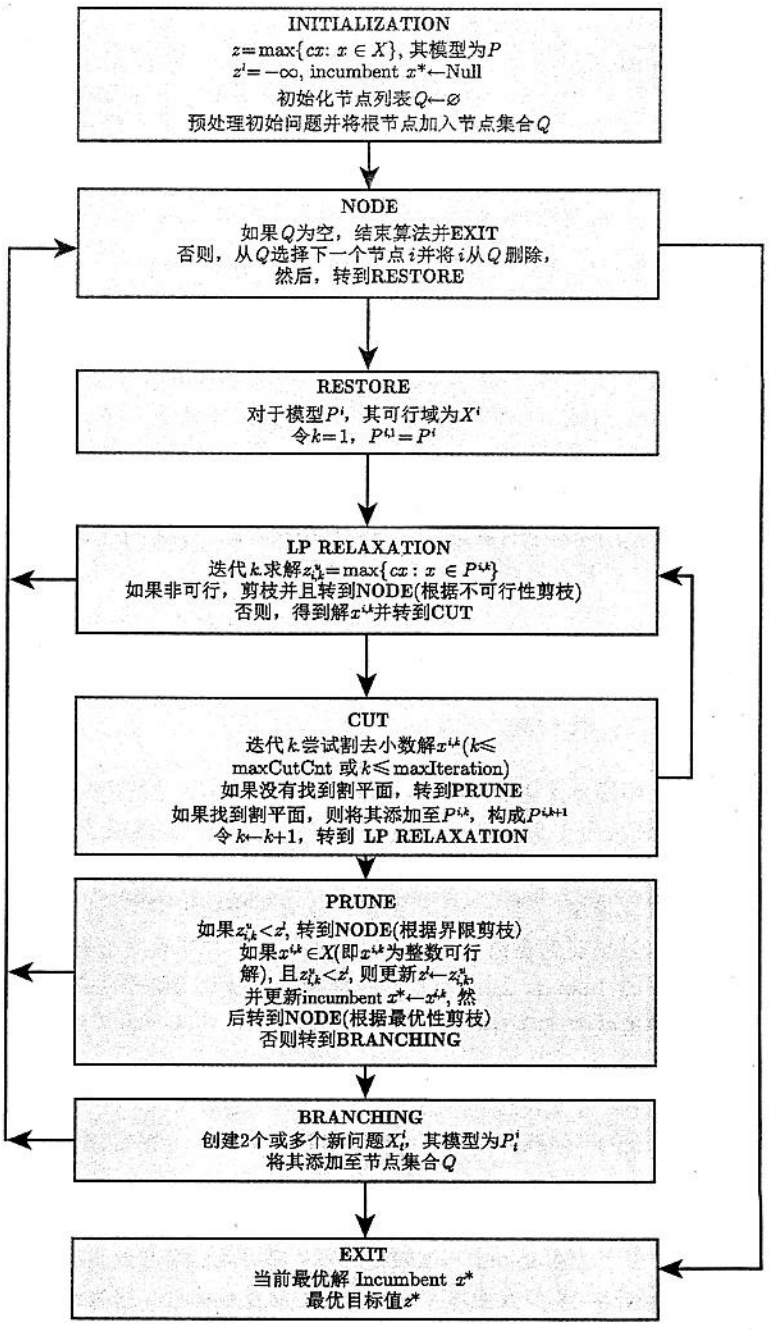


图 11.5　分支切割算法框图

上述算法框图给出了清晰的思路，参照上面的框图，我们总结出分支切割算法的详细

伪代码。伪代码对于代码实现可以提供更加直接的帮助。

Algorithm 7 Branch and Cut 算法

(***** INITIALIZATION *****)
初始化: $z = \max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{x} \in X\}$, 其模型为 P (P 是线性松弛)
$\underline{z} \leftarrow -\infty$
当前最优解 $\boldsymbol{x}^* \leftarrow$ Null
节点列表 $Q \leftarrow P$
while Q 非空 **do**
    (***** NODE *****)
    从 Q 中选择节点 i，并删除
    (RESTORE)
    获得节点 i 的模型，其可行域为 X^i
    (X^i 就是第 i 个节点整数规划原模型约束加上 cut 之后的模型)
    $k \leftarrow 1, P^{i,1} \leftarrow P^i$
    (***** LP RELAXATION *****)
    for $k \leqslant$ maxIteration **do**
        求解 $P^{i,k}$, 并令 $\bar{z}^{i,k} = \max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{x} \in P^{i,k}\}$
        if $P^{i,k}$ 非可行 **then**
            break (转到 **NODE**)
        else if $P^{i,k}$ 可行 **then**
            解 $\boldsymbol{x}^{i,k} \leftarrow$ 提取 $P^{i,k}$ 的解
            (***** CUT *****)
            尝试割去小数解 $\boldsymbol{x}^{i,k}$
            if 没有找到割平面 **then**
                (***** PRUNE *****)
                if $\bar{z}^{i,k} \leqslant z$ (此时可以根据界限剪枝) **then**
                    根据界限剪枝
                    break (转到 **NODE**)
                else if $\boldsymbol{x} \in X$ (也就是 $\boldsymbol{x}$ 是整数解) **then**
                    $\underline{z} \leftarrow \bar{z}^{i,k}$
                    incumbent $\boldsymbol{x}^* \leftarrow \boldsymbol{x}^{i,k}$
                    break (转到 **NODE**)
                else
                    (***** BRANCHING *****)
                    创建两个或更多新问题 X_t^i，其模型为 P_t^i
                    将 $\{P_t^i\}$ 添加至节点列表 $Q \leftarrow Q \cup \{P_t^i\}$

```
            end if
        else
            P^{i,k+1} ← 将割平面添加至 P^{i,k}
            k ← k + 1
        end if
    end if
  end for
end while
return 最优解 x*, 最优目标值 z*
```

11.4.2 分支切割算法：一个详细的例子

为了更直观地理解分支切割算法，我们以广义指派问题（Generalized Assignment Problem）为例，来大致进行一遍分支切割算法的过程，例子来源于文献（Wolsey，1998）。广义指派问题的模型如下：

$$\max \sum_{i=1}^{m}\sum_{j=1}^{n} c_{ij}x_{ij} \tag{11.37}$$

$$\sum_{j=1}^{n} x_{ij} \leqslant 1, \qquad \forall i = 1,2,\cdots,m \tag{11.38}$$

$$\sum_{i=1}^{m} a_{ij}x_{ij} \leqslant b_j, \qquad \forall j = 1,2,\cdots,n \tag{11.39}$$

$$x \in \{0,1\}^{m\times n} \tag{11.40}$$

其中，$m=10, n=5, \boldsymbol{b}=(91,87,109,88,64)^{\mathrm{T}}$，且

$$[c_{ij}] = \begin{bmatrix} 110 & 16 & 25 & 78 & 59 \\ 65 & 69 & 54 & 28 & 71 \\ 19 & 93 & 45 & 45 & 9 \\ 89 & 31 & 72 & 83 & 20 \\ 62 & 17 & 77 & 18 & 39 \\ 37 & 115 & 87 & 59 & 97 \\ 89 & 102 & 98 & 74 & 61 \\ 78 & 96 & 87 & 55 & 77 \\ 74 & 27 & 99 & 91 & 5 \\ 88 & 97 & 99 & 99 & 51 \end{bmatrix}, [a_{ij}] = \begin{bmatrix} 95 & 1 & 21 & 66 & 59 \\ 54 & 53 & 44 & 26 & 60 \\ 3 & 91 & 43 & 42 & 5 \\ 72 & 30 & 56 & 72 & 9 \\ 44 & 1 & 71 & 13 & 27 \\ 20 & 99 & 87 & 52 & 85 \\ 72 & 96 & 97 & 73 & 49 \\ 75 & 82 & 83 & 44 & 59 \\ 68 & 8 & 87 & 74 & 4 \\ 69 & 83 & 98 & 88 & 45 \end{bmatrix}$$

我们用分支定界算法来生成 lifted cover inequalities（该割平面的具体解释见 Wolsey，1998）。在该实例中，程序一共为根节点传了 3 次参数，每一次传参都设置最多添加 5 个割平面。对于其他节点，只传一次参数，每次至多添加 5 个割平面。操作完成之后，获得的分支切割树（Branch and Cut Tree）如图 11.6 所示。

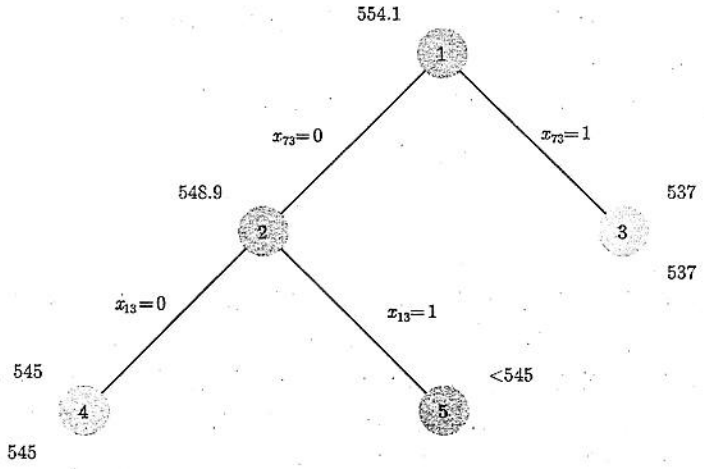


图 11.6 分支切割树

初始化：求解器的预求解部分直接删除了下面 5 个变量 $x_{11}, x_{32}, x_{62}, x_{72}, x_{65}$，因为 $a_{ij} > b_j$。例如 x_{11}, 对应的 $a_{11} = 95$, 而 $b_1 = 91$，这明显违背约束而导致问题不可行，因此直接将它们删除。

节点 1：节点 1 对应根节点。初始的线性规划的最优值为 595.6。循环 3 次添加割平面之后，一共添加了 14 个割平面，并且加强后的线性规划的最优值为 554.1，且对应的线性规划的解为

$$
\begin{aligned}
&x_{12} = 1, x_{23} = 0.5, x_{25} = 0.5, x_{31} = 0.76, x_{33} = 0.24,\\
&x_{43} = 0.5, x_{45} = 0.5, x_{52} = 0.74, x_{54} = 0.26, x_{61} = 1\\
&x_{73} = 0.5, x_{75} = 0.5, x_{82} = 1, x_{91} = 0.76, x_{94} = 0.24\\
&x_{10,1} = 0.24, x_{10,4} = 0.76
\end{aligned}
$$

其余的决策变量取值均为 0。

这里取最接近于 0.5 的其中一个决策变量 x_{73} 作为分支变量，构建两个新的子节点，子节点 2 对应 $x_{73} = 0$, 子节点 3 对应 $x_{73} = 1$。

节点 3：在子节点 3，算法调用了一次 cut 的函数，加入了 3 个割平面。获得了一个整数解，最优值为 537。因此把当前最优值（incumbent）的下界 $\underline{z}$ 和最优解 x^* 更新一下。另外，该点根据最优性被剪枝（Pruned by optimality）。

节点 2：在节点 2 处，算法调用了一次生成 cut 的函数，生成了一个割平面。这导致该点的加强的线性松弛问题的解为小数解，目标值为 548.9。选择 x_{13} 为分支变量，令 $x_{13} = 0$ 创建子节点 4，令 $x_{13} = 1$ 创建子节点 5。

节点 4：该子节点的子问题求到了整数解，目标值为 545。更新当前最优值和当前最优解。该点因为最优性被剪枝。

节点 5：该点的线性松弛问题的最优解没有超过当前最优值 545，因此根据 bound 将其剪枝（Pruned by bound）。

算法终止：未被查明的节点列表为空，因此得到了最优解

$$x_{12} = x_{23} = x_{31} = x_{43} = x_{52} = x_{61}$$
$$= x_{75} = x_{82} = x_{91} = x_{10,4} = 1$$

其余决策变量均为 0。

如果不设置在根节点调用产生割平面算法的次数，在根节点将会产生 18 个割平面。这样根节点的加强后的目标函数降为 546，并且之后的分支切割树也只有 3 个节点，节点数非常少。但是如果只用分支定界算法，要得到最优解，需要探索 4206 个节点才能获得最优解。这也说明了分支切割算法确实显著地减少了分支树的规模，探索的节点数也明显减少了。很多时候，分支切割算法确实是比分支定界算法更为有效，是更强大的算法。

11.5 Java 调用 CPLEX 实现分支切割算法求解 VRPTW

我们以 VRPTW 为例，来讲解如何使用分支切割算法求解 MIP。VRPTW 的标准模型见第 2.9 节。本节我们主要针对 VRPTW 的标准模型，设计分支策略（Branching Strategy）和割平面（Cutting Plane），然后将分支定界和割平面结合起来使用，组成本章介绍的分支切割。下面我们分别对分支定界和割平面进行单独介绍。

11.5.1 分支定界

关于分支定界的部分中上界、下界如何更新等细节问题，这里不再做详细阐述。本节主要介绍分支策略的设计。在分支定界的过程中，对于 BB tree 的每一个叶子节点，都对应一个 LP（该 LP 是原始 MIP 加上到该节点为止添加的所有分支约束和 Cut 之后，组成的新模型的线性松弛），我们求解该 LP，如果最优解为小数解，则需要进行分支操作。值得注意的是，分支不仅可以根据单个变量进行分支（整数变量和连续变量均可），还可以按照一些综合指标或者规则进行分支。本章的代码中，仅实现了针对单个决策变量进行了分支。但是我们在文中会介绍一些其他的分支策略。

回顾 VRPTW 的 MIP 模型，决策变量有两组，即为

（1）x_{ijk}：0-1 变量；

（2）s_i^k：连续型变量。

我们可以直接根据整数决策变量 x_{ijk} 进行分支，也可以针对连续变量 s_i^k 设计分支，还可以根据车辆数等综合指标分支。在这里，我们仅给出 3 种可选的分支策略：

（1）针对 x_{ijk} 进行分支；

（2）针对车辆数进行分支；

（3）针对资源窗口进行分支。

下面仅给出前 2 条分支策略的详细介绍。对于第 3 条分支策略，只给出粗略的描述。

可以将资源窗口分成若干更小的资源窗口，根据这些更小的资源窗口对模型进行分支，配合其他分支策略，加速整个求解过程，详见文献（Desaulniers et al., 2006）。

1. 对 x_{ijk} 进行分支

如果父节点的线性松弛后的 LP 的解不是整数可行解，我们任意选择一个取值为小数的 x_{ijk}，对其进行分支，创建两个子节点。其中左子节点加入约束 $x_{ijk}=0$，右子节点加入约束 $x_{ijk}=1$。之后就继续进行分支定界的迭代。也可以选择取值最接近 0.5 的 x_{ijk} 进行分支。

由于 x_{ijk} 是 0-1 变量，因此在 Java 调用 CPLEX 实现该分支操作的过程中，可以简单地通过改变 x_{ijk} 的上界或者下界来完成分支约束的添加。相应的函数为 `setLB()` 和 `setUB()`。当然，也可以通过构建新的约束，添加到子节点的 LP 中的方法实现。不过后者就需要特别注意，创建左子节点和右子节点时，一定要对父节点的 LP 做深拷贝，不能直接简单的赋值，否则有可能会导致左子节点添加完分支约束、右子节点由于没有做深拷贝而添加了两条互斥的分支约束，最终导致右子节点的 LP 不可行。

2. 对车辆数进行分支

除了对单个的决策变量进行分支，我们还可以对一些综合指标进行分支。使用的总车辆数就是一个非常有用的指标。这里，我们只关注单个弧段 (i,j) 上的车辆数。弧段 (i,j) 上的车辆数的总和为

$$f_{ij}=\sum_{k\in K}x_{ijk} \tag{11.41}$$

不难得出，弧段 (i,j) 上使用的车辆数一定是整数，否则该解必然不是可行解。假设 $f_{ij}=n_0$，n_0 为小数。则我们可以根据该信息创建两个子节点，同时也相应地创建 2 个互斥的分支约束

$$f_{ij}=\sum_{k\in K}x_{ijk}\leqslant\lfloor n_0\rfloor \tag{11.42}$$

$$f_{ij}=\sum_{k\in K}x_{ijk}\geqslant\lceil n_0\rceil \tag{11.43}$$

并将其分别添加到左右子节点中。

11.5.2 割平面

分支切割算法的另外一个重要部分就是割平面。割平面的作用就是在子节点处寻找到更紧的约束，添加到模型中，进而更好地逼近该节点 LP 的可行域中所有整数可行解构成的凸包。需要特别指出的是，这些约束必须要保证不会割去任何整数可行解。根据之前的介绍，割平面的加入，很多时候可以减少分支的次数，从而减少探索的节点的个数，进而加快求解速度。

事实上，分支定界算法的过程中加入的分支约束，就可以看作一种特殊的割平面。而分支切割算法中，只是在分支时，除了加入分支约束，还需要加入额外的 Cut。

本章仅采用最直观、最易理解的一种割，即 k-路径割平面（k-path Cuts）。

k-路径割平面是基于下面的观察提出的。给定一个顾客点的子集 $S \subset V$，我们可以评估出，服务完这些顾客最少需要的车辆数 $k(S)$。因此，在最优解中，对于子集 S 组成的部分图而言，进入集合 S 的车辆数必定大于或等于 $k(S)$。因此对于 VRPTW （和 CVRP）而言，我们就可以给出下面的有效不等式（Valid Inequality），或者说 Cuts：

$$X(S) = \sum_{(i,j)\in \text{in}(S)} x_{ij} \geqslant k(S) \tag{11.44}$$

其中,$(i,j) \in \text{in}(S)$ 表示进入集合 S 的所有弧，$X(S)$ 表示当前解中进入集合 S 的流量的总和，也就是进入集合 S 的车辆数。关于如何估计 $k(S)$，这里给出一个非常直观的方法。我们用集合 S 中的所有顾客的总需求，除以车辆的容量，取上界即可，即

$$k(S) = \left\lceil \frac{d(S)}{Q} \right\rceil = \left\lceil \frac{\sum_{i\in S} q_i}{Q} \right\rceil \tag{11.45}$$

上述评估方法仅仅是给出了 $k(S)$ 的一个上界，实际上该上界是不够紧的，有一些研究针对此做了一些加强，提出了更紧的 $k(S)$ 的上界，详见文献 Baldacci et al., 2008 和 Costa et al., 2019。

本书提供了上述内容的完整代码实现，详见附配资源。

11.6 Python 调用 Gurobi 实现分支切割算法求解 VRPTW 完整代码

我们也提供了 Python 版本的分支切割算法的代码，包括手动实现分支定界以及割平面的版本和使用 callback 添加割平面的版本。同样，我们选取取值最接近 0.5 的决策变量 x 进行分支，在每个子节点处，最多添加 5 条 k-路径割平面。

我们取 C101 中前 60 个客户点作为测试算例，设置车辆数为 8，对比分支定界算法和分支切割算法的求解效果。如图 11.7 所示，分支切割算法的迭代次数较少，探索的节点数也更少。

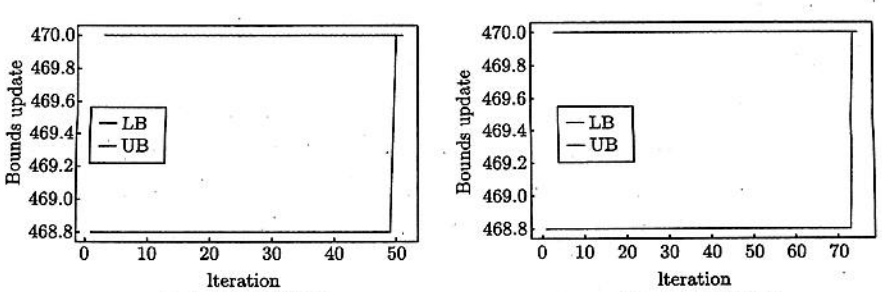


图 11.7 分支切割算法和分支定界算法的 UB 和 LB 更新比较 (C101-60)

在 callback 版本的代码中，首先要注意，一定要将参数 PreCrush 设置为 1，语法为 model.Params.PreCrush=1。因为在 Gurobi 中，想要添加自己设计的 Cut，必须要设置该参数。构造自己设计的 Cut 之前，首先需要在 callback 函数中判断 where 等于 GRB.Callback.MIPNODE，也就是我们要在 BB tree 的节点处添加 Cut。然后我们判断该节点的线性松弛模型是有最优解的，即用语句 model.cbGet(GRB.Callback.MIPNODE_STATUS) 得到该点的求解状态，当状态为 GRB.OPTIMAL 时，我们就可以添加 Cut 了。此时，先通过调用函数 cbGetNodeRel 获得当前节点的松弛解，然后根据松弛解和外部变量 model._vars 构造 Cut 的左端线性表达式，最后调用函数 cbCut 将该 Cut 添加到当前模型中。注意，函数 cbCut 的参数中没有 name 这个参数。按照上述操作，就完成了使用 callback 构造用户自己设计的 Cut。

11.7 Java 调用 CPLEX 实现分支切割算法求解 CVRP: 回调函数添加割平面

本节提供了一种基于求解器提供的 callback 函数来实现分支切割的方法。这种方法省去了自己实现分支定界算法的过程，便于研究者将主要精力集中在如何构建 Cut 上，并且实现起来更容易。

我们以 Capacitated Vehicle Routing Problem （CVRP）为例，介绍结合 callback 函数的分支切割算法的实现。本代码通过简单的修改之后，也可以用于求解 VRPTW。接下来，我们首先给出 CVRP 的基本模型，然后给出割平面的详细介绍，最后附以完整的代码。

11.7.1 CVRP 的基本模型

CVRP 可以看作 VRPTW 的一个松弛版本。只要将 VRPTW 中的时间窗约束去掉，VRPTW 就退化成了 CVRP。在本节中，我们考虑所有的车都是同质的。

本节中用到的模型来自文献（Toth and Vigo, 2014），模型所需的符号定义如下。客户集为 $N=\{1,2,\cdots,n\}$，其中每个客户 i 都具有已知的非负需求 q_i。中央仓库，即 depot，用 0 点表示。K 辆相同的车辆被停放在 depot，每辆车的容量为 Q。

设 $G=(V,E)$ 是一个完全图，其中，$V=\{0\}\cup N$，且 $E=\{(i,j):i,j\in V,i\neq j\}$，这里，我们考虑 E 是无向的。c_{ij} 是每个边 $(i,j)\in E$ 的成本。

给定一个客户集 $S\subseteq N$，设 $q(S)=\sum\limits_{i\in S}q_i$ 表示集合的总需求量。$\delta(S)$ 表示在 S 中只有一个端点的边的集合。$e\in E$ 为所有弧的集合。当考虑单顶点 $i\in V$ 时，写作 $\delta(i)$。$A(S)$ 表示 S 中的所有边。

我们用 $\gamma(S)$ 表示需要为 S 中所有客户提供服务车辆的最小数量，它可以被当作装箱问题来计算。我们通常取 $\gamma(S)=\lceil q(S)/Q\rceil$。

基于上面的介绍，我们给出 CVRP 的基本模型如下：

$$\min \quad \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \tag{11.46}$$

$$\text{s.t.} \quad x(\delta(i)) = 2, \qquad \forall i \in N \tag{11.47}$$

$$x(\delta(0)) = 2\,|K| \tag{11.48}$$

$$x(\delta(S)) \geqslant 2\gamma(S), \quad \forall S \subseteq N, \forall S \neq \varnothing \tag{11.49}$$

$$x_e \in \{0,1,2\}, \qquad \forall e \in \delta(0) \tag{11.50}$$

$$x_e \in \{0,1\}, \qquad \forall e \in E\text{除去}\delta(0) \tag{11.51}$$

其中，约束 (11.47) 和 (11.48) 保证了每个客户点均被访问一次，有且仅有 K 辆车从 depot 出发并回到 depot。约束 (11.49) 则保证了容量及回路约束（Subtour Elimination Constraints, SECs）。

由于约束 (11.49) 数量非常庞大，在建模的时候就全部列举并且加入非常耗时。因此我们用惰性约束（lazy constraint）的方式添加该约束。也就是说，在 CVRP 的分支切割算法代码中，约束 (11.49) 将先被松弛掉，在松弛后的模型解出整数解时，做出当前解是否违背该约束的判断，再以惰性约束的方式加入模型中重新求解。同时，在模型解出小数解时也可以用同样的方式加入一些 user cut 到模型中，使模型的解空间缩小来进行加速。需要注意的是，在某一节点加入过多的用户定义割平面（user cut）也会占用不少的处理时间，影响模型的求解速度。但我们不用为此担心，CPLEX 和 Gurobi 均提供了十分好用的 callback 函数，可以直接用于用户自定义割平面和惰性约束的设置，在模型求解的过程中这些约束会自动选择合适的时机加入模型中，省去了手动进行分支定界的过程，十分方便。

11.7.2 割平面

本节中所采用的 Cut 依然是上节所提到的 k 路径割平面，即约束 (11.49)。其中约束右端项中对于最少所需车辆数的估计，可以直接用集合 S 中的所有顾客的总需求除以车辆的容量，取上界即可。

1. 单个割平面的生成

对于松弛模型求得的解，需要使所有的客户集子集 $S \subseteq N$ 都满足容量及回路约束，才能得到最优解。所以要判断当前解是否违背约束，需要快速找出客户集 S，才能进一步生成 Cut 加入模型中。一种简单的寻找方式如下。

（1）首先找到当前解中取值最大的一条边 $(i,j) \in E$，将对应的客户点 i,j 合并得到 S。

（2）再依次寻找与合并后的 S 相连取值最大的边，并将对应的点加入 S，直到找不到任意一条取值大于 0 的边与 S 相连。

（3）将 S 中所有点到 depot 和其他不在 S 中的点对应的边的取值相加得到 $x(\delta(S))$，并求得 S 所需的最小车辆数 $\lceil q(S)/Q \rceil$。

（4）判断约束 (11.49) 是否满足，若不满足则将生成的约束加入模型中。

2. CPLEX 中 callback 函数的调用方法

CPLEX 提供了 IloCplex.UserCutCallback 和 IloCplex.LazyConstraintCallback 两个类。用户可以通过自定义类来继承这两个父类来完成 callback 函数的设置。

本节中调用 UserCutCallback 加 Cut 的代码如下。继承后的类会自动生成一个 main() 函数，在 main() 函数中即可调用 Cut 的生成函数，并将生成的 Cut 通过 addLocal() 函数加入模型中。

UserCutCallback

```
1        public static class Callback extends IloCplex.UserCutCallback{
2        Cut cut;
3        ArrayList<IloNumExpr> cutLhs;
4        ArrayList<Integer> cutRhs;
5        IloNumVar[] x;
6        IloCplex ilcplex;
7        int nCus;
8        Instance instance;
9        Callback(IloNumVar[] x0,IloCplex ilcplex0,int nCus0,Instance
       instance0){
10               x=x0;
11               ilcplex=ilcplex0;
12               nCus=nCus0;
13               instance=instance0;
14       }
15
16       public void main() throws IloException{
17               double[] xSol = getValues(x);
18               //生成Cut
19               cut = makecuts(x, xSol, nCus, ilcplex, instance);
20               //添加Cut
21               cutLhs = cut.getLhsExprs();
22               cutRhs= cut.getRight();
23               for(int i = 0; i< cutLhs.size(); i++) {
24                       addLocal(ilcplex.ge(cutLhs.get(i), cutRhs.get(i)));
25               }
26       }
27 }
```

调用 LazyConstraintCallback 的代码 UserCutCallback 几乎相同，其中需要注意的是生成的 Cut 是通过 add() 函数加入模型中，而不是 addLocal() 函数。代码如下：

LazyConstraintCallback

```
1   public static class LazyCallback extends IloCplex.
    LazyConstraintCallback {
2           Cut cut;
3           ArrayList<IloNumExpr> cutLhs;
4           ArrayList<Integer> cutRhs;
5           IloNumVar[] x;
6           IloCplex ilcplex;
7           int nCus;
8           Instance instance;
9           LazyCallback(IloNumVar[] x0,IloCplex ilcplex0,int nCus0,
    Instance instance0){
10                  x=x0;
11                  ilcplex=ilcplex0;
12                  nCus=nCus0;
13                  instance=instance0;
14          }
15
16          public void main() throws IloException{
17                  double[] xSol = getValues(x);
18                  cut = makecuts(x, xSol, nCus, ilcplex, instance);
19                  cutLhs = cut.getLhsExprs();
20                  cutRhs= cut.getRight();
21                  for(int i = 0; i< cutLhs.size(); i++) {
22                     add(ilcplex.ge(cutLhs.get(i), cutRhs.get(i)));
23                  }
24          }
25  }
```

在完成模型的其他基本约束及目标函数定义后，将两个定义好的类加入模型中，然后进行求解即可。代码如下：

LazyConstraintCallback

```
1   ilcplex.use(new Callback(x,ilcplex,nodeNum,instance));
2   ilcplex.use(new LazyCallback(x,ilcplex,nodeNum,instance));
```

本节的完整代码见附配资源。

第12章　拉格朗日松弛

拉格朗日松弛（Lagrangian Relaxation）是一个非常重要的算法。该算法在运筹优化领域通常与分支定界算法、分支切割算法、分支定价算法等联合使用，一般用来为算法提供较好的上界或者下界，以加快算法收敛。本章我们首先介绍最优性（Optimality）和松弛（Relaxation）的相关理论，然后详细介绍拉格朗日松弛的原理、伪代码和代码实现。本章的大部分内容均参考自文献（Wolsey，1998）第 2 章和第 10 章，感兴趣的读者可以参考相应章节阅读完整内容。另外，本章中涉及较多的定理，为了提高可读性，我们仅给出了部分定理的证明。

12.1　最优性和松弛

给定一个整数规划（Integer Programming，IP）或者组合优化问题（Combinatorial Optimization Problem，COP）模型

$$z = \max\{c(\boldsymbol{x}) : \boldsymbol{x} \in X \subseteq \mathbb{Z}^n\} \tag{12.1}$$

我们如何能够证明一个给定的解 $\boldsymbol{x}^*$ 是最优解呢？为了证明这一点，我们需要给出一些最优性的判定条件，这能够为我们的 IP 算法的停止条件提供一些帮助。

一个比较基本的想法就是，我们找到一个下界 $\underline{z} \leqslant z$ 以及一个上界 $\bar{z} \geqslant z$，使得 $\underline{z} = \bar{z} = z$。这意味着在实际的算法迭代中，我们将会找到一个目标函数 UB 的递减序列

$$\bar{z}_1 > \bar{z}_2 > \cdots > \bar{z}_s \geqslant z$$

以及一个下界的递增序列

$$\underline{z}_1 < \underline{z}_2 < \cdots < \underline{z}_t \leqslant z$$

当上界和下界的差值小于我们给定的一个容差界限 ϵ（$\epsilon > 0$, 例如 0.001）时，算法就可以停止了。如图 12.1所示，也就是

$$\bar{z}_s - \underline{z}_t \leqslant \epsilon$$

因此，我们需要找出些能够产生这样的上界和下界的方法。

如果说能够找到一些非常好的界限，这将为我们的算法收敛起到很明显的帮助。从理论上来讲，找到一些比较好的上界或者下界，也是比较好的理论贡献。有些时候，虽然说

得到最优解很困难，大规模问题也较难求解，但是如果能通过理论推导，得出比较紧的界限，可能对算法的收敛有较大的帮助。而拉格朗日松弛有时就会用于得到原问题的一个边界。

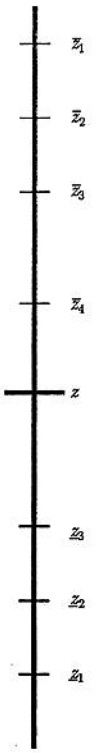


图 12.1　整数规划的解的界限

12.1.1　原始边界

我们知道，对于 LP 而言，一个原问题（Primal Problem），会有与之对应的对偶问题（Dual Problem）。如果原问题为 max，那么对偶问题就为 min。并且对偶问题 min 的任意一个可行解，都是原问题 max 的一个上界。这是弱对偶性定理。另外，如果 LP 的原问题和对偶问题的可行域均为非空，则原问题和对偶问题的最优解的最优值是一样的（强对偶性定理）。因此，对偶理论在推导界限方面，有非常大的作用。原问题的解的信息，就可以为原问题提供界限。

具体来讲，每一个可行解 $\boldsymbol{x}^* \in X$ 都是原问题的一个下界 $\underline{z} = c(\boldsymbol{x}^*) \leqslant z$，我们称这种界限为原始边界（Primal Bounds），简称原界。这是我们唯一知道的能够获得下界的方法。对于一些 IP 而言，找到一个可行解是非常容易的，但是关键的问题是如何找到比较高质量的可行解。比如说 TSP，如果任意两个节点之间都可以直接到达，那么交换任意两个节点的访问顺序，就可以得到一个可行解。我们通过评估这个可行解，就可以得到全局最优

值 z 的一个原界。另外一些比较难的 IP，我们就需要更复杂的获得可行解的办法，比如用启发式算法获得可行解，从而获得下界。

12.1.2 对偶边界

获得 max 问题的上界（或者 min 问题的下界）是非常困难的事情。这些界限叫作对偶边界（Dual Bounds）。称其为对偶边界的原因是，对于 max 问题，其对偶问题（min）的任意一个可行解就是原问题（max）的一个上界（因此通过求解对偶问题，得到原问题的上界是一个可选的方法）。基于此，我们将 max 问题的上界，或者 min 问题的下界称为对偶边界。获得对偶边界的最重要的方法就是“松弛”，其主要思想就是，用一个比较简单的优化模型来替代一个比较困难的 max（min）IP，这个简单的优化模型的最优值大于或等于（小于或等于）原来 IP 的最优值 z。为了让松弛问题有如上所述的性质，我们可以做下面两类操作。

（1）松弛问题扩大了可行解的集合，因此是在一个更大的集合上做优化；

（2）将原来的 max（min）目标函数替换为一个取值永远大于或等于（小于或等于）原目标函数的函数。

定义 12.1.1 （Wolsey，1998）问题（RP）$z^R = \max\{f(\boldsymbol{x}) : \boldsymbol{x} \in T \subseteq \mathbb{R}^n\}$ 是原问题（IP）$z = \max\{c(\boldsymbol{x}) : \boldsymbol{x} \in X \subseteq \mathbb{R}^n\}$ 的一个松弛，当满足下列条件：

（1）$X \subseteq T$；

（2）$f(\boldsymbol{x}) \geqslant c(\boldsymbol{x}), \forall \boldsymbol{x} \in X$。

定理 12.1.1 （Wolsey，1998）如果 RP 是 IP 的一个松弛，则 $z^R \geqslant z$。

证明： 如果 $\boldsymbol{x}^*$ 是 IP 的一个最优解，则 $\boldsymbol{x}^* \in X \subseteq T$，且 $z = c(\boldsymbol{x}^*) \leqslant f(\boldsymbol{x}^*)$。由于 $\boldsymbol{x}^* \in T$，$f(\boldsymbol{x}^*)$ 是 z^R 的一个下界，因此 $z \leqslant f(\boldsymbol{x}^*) \leqslant z^R$。

12.2 对 偶

由于对偶理论（Duality Theory）是获得线性规划（max 问题）的上界的一个标准方法，因此我们就要问，我们有没有可能获得整数规划的对偶呢？虽然整数规划不能像线性规划一样，可以直接写出对偶问题的模型，但是我们可以通过其他方法获得一些信息。对偶的一个很好的性质就是，对偶问题的任意一个可行解，就是原问题目标函数值 z 的一个上界（max 问题）。

定义 12.2.1 （Wolsey，1998）假设有 2 个问题

$$(\text{IP}): \quad z = \max\{c(\boldsymbol{x}) : \boldsymbol{x} \in X\}$$

$$(D): \quad w = \min\{w(\boldsymbol{u}) : \boldsymbol{u} \in U\}$$

当 $c(\boldsymbol{x}) \leqslant w(\boldsymbol{u})$，对所有 $\boldsymbol{x} \in X$ 和 $\boldsymbol{u} \in U$ 都成立时，IP 和 D 是一对弱对偶对（weak-dual pair）。当 $z = w$ 时，它们是一对强对偶对（strong-dual pair）。

对偶问题相比于松弛的方法的好处在于，任意一个对偶问题的可行解都是原问题的最优值 z 的一个上界，而对于原问题 IP 而言，必须要求解 IP 的线性松弛，得到最优解，才能获得一个上界。二者相比，求解对偶问题，得到一个可行解显然更容易。对偶问题的这个优势是非常棒的。但问题是，IP 的对偶真的存在吗？根据之前的介绍，我们知道，IP 的强对偶我们不确定其存在与否，但幸运的是，IP 的弱对偶却是存在的，且可以直接写出来。

不难得出，原问题 IP 的线性松弛的对偶可以为原问题提供一个弱对偶。因为 $z^R \geqslant z^{\mathrm{IP}}$，而线性松弛问题的对偶的任意可行解（目标值用 z^{DR} 表示）是原问题 IP 的线性松弛的上界，因此 $z^{\mathrm{DR}} \geqslant z^R \geqslant z^{\mathrm{IP}}$。

定理 12.2.1 （Wolsey，1998）整数规划 $z = \max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}, \boldsymbol{x} \in \mathbb{Z}_+^n\}$ 和线性规划 $w^{\mathrm{LP}} = \min\{\boldsymbol{u}^{\mathrm{T}}\boldsymbol{b}, \boldsymbol{u}^{\mathrm{T}}\boldsymbol{A} \geqslant \boldsymbol{c}, \boldsymbol{u} \in \mathbb{R}_+^m\}$ 组成一个弱对偶对。

根据文献（Wolsey，1998）的命题 2.3，不难得出，对偶问题有时候可以辅助我们验证解的最优性。

定理 12.2.2 （Wolsey，1998）假设 IP 和 D 是一对弱对偶对，则

（1）如果 D 是无界的，则 IP 是非可行的；

（2）如果 $\boldsymbol{x}^* \in X$，且 $\boldsymbol{u}^* \in U$ 满足 $c(\boldsymbol{x}^*) = w(\boldsymbol{u}^*)$，则 $\boldsymbol{x}^*$ 是 IP 的最优解，且 $\boldsymbol{u}^*$ 是 D 的最优解。

12.3 拉格朗日松弛

12.3.1 拉格朗日松弛介绍

介绍完上面的基础理论，下面来正式介绍拉格朗日松弛。首先，我们假设有下面的整数规划（IP）。

$$
\begin{aligned}
z = \max \quad & \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} \\
\text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b} \\
& \boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d} \\
& \boldsymbol{x} \in \mathbb{Z}_+^n
\end{aligned}
\tag{IP}
$$

假如只考虑约束 $\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}$，模型非常容易求解，而加入了约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 之后，就非常难求解。如果说我们将比较难求解的约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 松弛掉，那么剩余的问题相对于原来的 IP 就容易多了。这种情况非常常见，例如 TSP 中消除子环路的约束比较复杂，如果将其删除，剩余的部分将变成一个指派问题，非常容易求解。或者既有整数变量、也有连续变量的模型中，将整数变量参与的约束松弛掉，问题也会变得非常容易求解。

但是，如果直接将比较复杂的约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 删除，那得到的界限就会非常弱，因为忽略了很重要的约束。不过可以通过拉格朗日松弛来改善这个缺陷。

下面考虑 IP：

$$
z = \max \quad \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}
$$

$$\text{s.t.} \quad \boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$$
$$\boldsymbol{x} \in X$$

其中，$\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 是 m 个比较复杂的约束。

注意：这里的 $\boldsymbol{x} \in X$ 指的是 $\boldsymbol{x}$ 落在 $\boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}$ 和 $\boldsymbol{x} \in \mathbb{Z}_+^n$ 共同约束的可行域里。

对于任意的 $\boldsymbol{u} = (u_1, u_2, \cdots, u_m), u_i \geqslant 0, \forall i = 1, 2, \cdots, m$，我们定义下面的问题：

$$\text{IP}(\boldsymbol{u}) \qquad z(\boldsymbol{u}) = \max \quad \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} + \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x})$$
$$\text{s.t.} \qquad \boldsymbol{x} \in X$$

定理 12.3.1 （Wolsey，1998）对于任意的 $\boldsymbol{u} \geqslant \boldsymbol{0}$，问题 $\text{IP}(\boldsymbol{u})$ 是问题 IP 的一个松弛问题。

证明：回忆之前讲过的，问题 $\text{IP}(u)$ 是问题 IP 的一个松弛，需要满足下面两个条件：

（1）两者的可行域至少要一样大。上述命题中，因为 $\{\boldsymbol{x} : \boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}, \boldsymbol{x} \in X\} \subseteq X$，所以两者的可行域满足该条件。

（2）对于处在 IP 的可行域中的任意可行解，对应的 $\text{IP}(\boldsymbol{u})$ 的目标函数，要大于或者等于 IP 的目标函数。由于 $\boldsymbol{u} \geqslant \boldsymbol{0}$，且 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$，因此对于任意的 $\boldsymbol{x} \in X$，$\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} + \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}) \geqslant \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}$。问题 $\text{IP}(\boldsymbol{u})$ 和 IP 同时满足上面两条，因此命题得证。

可以发现，对于每一个复杂约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$，处理它们的方式就是将它们加入目标函数中，再乘以一个权重 $\boldsymbol{u}$，构成一个惩罚项 $\boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x})$。这个惩罚因子 $\boldsymbol{u}$ 其实就是约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 的影子价格（Shadow Price）或者对偶变量（Dual Variable），我们称其为拉格朗日乘子（Lagrange Multiplier）。

将 $\text{IP}(\boldsymbol{u})$ 称为原问题 IP 的参数为 $\boldsymbol{u}$ 的拉格朗日松弛的子问题。由于 $\text{IP}(\boldsymbol{u})$ 是 IP 的一个松弛，因此 $z(\boldsymbol{u}) \geqslant z$，于是就获得了 IP 的一个上界。我们的目标是找到最好的上界，也就是找到最佳的 $\boldsymbol{u}$ 值，使得拉格朗日松弛的目标函数尽可能小，这样才能更好地逼近 IP 的目标函数。

12.3.2 拉格朗日对偶问题

为了获得最佳的 $\boldsymbol{u}$，我们需要解决下面的拉格朗日对偶问题（Lagrangian Dual Problem）：

$$\text{(LD)} \qquad w_{\text{LD}} = \min\{z(\boldsymbol{u}) : \boldsymbol{u} \geqslant \boldsymbol{0}\}$$

我们观察到，当 m 个复杂约束都取到了等号，也就是 $\boldsymbol{D}\boldsymbol{x} = \boldsymbol{d}$ 时，对应的拉格朗日乘子 $\boldsymbol{u}$ 就可以变成无符号限制的，即此时 $\boldsymbol{u} \in \mathbb{R}^m$。相应地，拉格朗日对偶问题就变成了

$$w_{\text{LD}} = \min_{\boldsymbol{u}} z(\boldsymbol{u})$$

求解上面的拉格朗日松弛 $\text{IP}(\boldsymbol{u})$ 有时可能会得到原问题 IP 的最优解。

定理 12.3.2 （Wolsey，1998）如果 $\boldsymbol{u} \geqslant \mathbf{0}$，且有

（1）$\boldsymbol{x}(\boldsymbol{u})$ 是 IP$(\boldsymbol{u})$ 的一个最优解；

（2）$\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}) \leqslant \boldsymbol{d}$；

（3）对于所有的 $u_i > 0$，都有 $[\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u})]_i = d_i$（互补松弛性），

则 $\boldsymbol{x}(\boldsymbol{u})$ 是 IP 的最优解。

证明：根据定理第（1）条，我们有 $w_{\mathrm{LD}} \leqslant z(\boldsymbol{u}) = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u}) + \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}))$。根据定理第（3）条，我们有 $\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u}) + \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}(\boldsymbol{u})) = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u})$。根据定理第（2）条，我们有 $\boldsymbol{x}(\boldsymbol{u})$ 是 IP 的一个可行解，因此 $\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u}) \leqslant z$。因此我们有 $w_{\mathrm{LD}} \leqslant \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u}) + \boldsymbol{u}^{\mathrm{T}}(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}(\boldsymbol{u})) = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}(\boldsymbol{u}) \leqslant z$。但是由于 $w_{\mathrm{LD}} \geqslant z$，因此此时 $w_{\mathrm{LD}} = z$，所以得到，$\boldsymbol{x}(\boldsymbol{u})$ 是 IP 的最优解。

注意到如果约束 $\boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}$ 都取到了等号，那么条件（3）就会自动成立。且此时如果 IP$(\boldsymbol{u})$ 的一个最优解同时也是 IP 的可行解，那么该解同时也是 IP 的最优解。

这里来直观解释互补松弛性。互补松弛性定理的内容是，若原问题和对偶问题均得到了最优解，则原问题第 i 个约束的松弛变量 s_i 与其对偶变量 y_i 的乘积为 0，即 $s_i y_i = 0$。而松弛变量为 $s_i = d_i - (\boldsymbol{D}\boldsymbol{x})_i$，因此有 $y_i(d_i - (\boldsymbol{D}\boldsymbol{x})_i) = 0$。

互补松弛性并不难理解，约束 $(\boldsymbol{D}\boldsymbol{x})_i \leqslant d_i$ 是第 i 种资源的限制，该资源是有限的。如果该资源是紧缺的，那么该资源一定供不应求，也就是在最优解中，一定有 $(\boldsymbol{D}\boldsymbol{x})_i = d_i$，即该种资源全部都被使用了。此时，如果资源 i 能够再增加一些，目标函数还能继续增大。这就说明，该资源的边际收益或者影子价格就大于 0。影子价格反映了单位资源的增加会给目标函数带来的收益。此时，影子价格和第 i 个松弛变量满足 $s_i y_i = 0$。

反之，如果资源 i 是不紧缺的或者是充足的，那么最优解一般会满足 $(\boldsymbol{D}\boldsymbol{x})_i < d_i$，该资源 i 会剩余 $d_i - (\boldsymbol{D}\boldsymbol{x})_i$。因此剩余变量为 $s_i = d_i - (\boldsymbol{D}\boldsymbol{x})_i > 0$。既然该资源是充足的、剩余的，那么再增加该资源的供应，也不会改善目标函数，因此其影子价格就为 0，即 $y_i = 0$。此时，亦有 $s_i y_i = 0$。这就是互补松弛性的直观理解。

下面我们以一个经典问题的整数规划模型来介绍拉格朗日松弛的用法。

12.3.3 拉格朗日松弛应用案例：无容量限制的设施选址问题

我们以无容量限制的设施选址问题（Uncapacitated Facility Location Problem，UFLP）为例来介绍拉格朗日松弛的应用。

首先，我们给出 UFLP 的问题描述：给定一个可选设施点的集合 $N = \{1, 2, \cdots, n\}$ 和一个客户点的集合 $M = \{1, 2, \cdots, m\}$。假设每一个设施点都对应固定成本 f_j，即如果设施 j 被开通，则会产生固定成本 f_j。并且，如果顾客 i 的需求需要被设施 j 满足，则会相应地产生收益 c_{ij}。UFLP 就是要决策开通哪些设施点，使得所有客户的需求都被满足，并且要最大化总净收益（等于总收益减去总成本）。

首先，我们引入下面的决策变量：

(1) y_j：如果设施 $j \in N$ 被使用，则 $y_j = 1$，否则 $y_j = 0$；

(2) x_{ij}：顾客点 i 的需求被设施 j 满足的比例。

下面我们给出 UFLP 的模型。

$$\text{(IP)} \quad \max\ z = \sum_{i\in M}\sum_{j\in N} c_{ij}x_{ij} - \sum_{j\in N} f_j y_j \tag{12.2}$$

$$\text{s.t.} \quad \sum_{j\in N} x_{ij} = 1, \qquad \forall i \in M \tag{12.3}$$

$$x_{ij} \leqslant y_j, \qquad \forall i \in M, \forall j \in N \tag{12.4}$$

$$x \in \mathbb{R}^{|M|\times|N|}, y \in \mathbb{B}^{|N|} \tag{12.5}$$

我们将需求约束 (12.3) 对偶上去（松弛掉），则目标函数变化为

$$\sum_{i\in M}\sum_{j\in N} c_{ij}x_{ij} - \sum_{j\in N} f_j y_j + \sum_{i\in M} u_i\left(1 - \sum_{j\in N} x_{ij}\right) \tag{12.6}$$

$$= \sum_{i\in M}\sum_{j\in N} (c_{ij} - u_i)\, x_{ij} - \sum_{j\in N} f_j y_j + \sum_{i\in M} u_i \tag{12.7}$$

因此拉格朗日松弛即为

$$\text{IP}(\boldsymbol{u}) \quad \max\ z(\boldsymbol{u}) = \sum_{i\in M}\sum_{j\in N} (c_{ij} - u_i)\, x_{ij} - \sum_{j\in N} f_j y_j + \sum_{i\in M} u_i \tag{12.8}$$

$$\text{s.t.} \quad x_{ij} \leqslant y_j, \qquad \forall i \in M, \forall j \in N \tag{12.9}$$

$$x \in \mathbb{R}^{|M|\times|N|}, y \in \mathbb{B}^{|N|} \tag{12.10}$$

我们发现，对于约束而言，约束 (12.9) 和 (12.10) 都含有 $\forall j \in N$，因此可以将模型 $\text{IP}(\boldsymbol{u})$ 按照下标 $j \in N$ 分解成 N 个子问题，即每一个候选地址都对应一个子问题。

因此有 $z(\boldsymbol{u}) = \sum_{j\in N} z_j(\boldsymbol{u}) + \sum_{i\in M} u_i$，其中子问题为

$$\text{IP}(\boldsymbol{u})_j \quad z_j(\boldsymbol{u}) = \max \sum_{i\in M} (c_{ij} - u_i)\, x_{ij} - f_j y_j \tag{12.11}$$

$$\text{s.t.} \quad x_{ij} - y_j \leqslant 0, \qquad \forall i \in M \tag{12.12}$$

$$x_{ij} \geqslant 0, y_j \in \{0,1\}, \qquad \forall i \in M \tag{12.13}$$

子问题 $\text{IP}_j(\boldsymbol{u})$ 是非常容易求解的。如果 $y_j = 0$，则 $x_{ij} = 0, \forall i \in M$，因此目标函数就为 0。如果 $y_j = 1$，所有满足 $c_{ij} - u_i \geqslant 0$ 的顾客将会把全部需求都交给设施 j 满足。此时目标函数值等价为 $\sum_{i\in M} \max[c_{ij} - u_i, 0] - f_j$。因此，有

$$z_j(\boldsymbol{u}) = \max\left\{0, \sum_{i\in M} \max[c_{ij} - u_i, 0] - f_j\right\}$$

下面以一个具体的算例来解释该问题。考虑一个 $m=6,n=5$ 的 UFLP。其中，m 是客户的数量，n 是候选设施的数量。固定费用为 $\boldsymbol{f}^{\mathrm{T}}=[2,4,5,3,3]$，且收益矩阵为

$$\{c_{ij}\}=\begin{bmatrix}6&2&1&3&5\\4&10&2&6&1\\3&2&4&1&3\\2&0&4&1&4\\1&8&6&2&5\\3&2&4&8&1\end{bmatrix}$$

这里取 $\boldsymbol{u}^{\mathrm{T}}=[5,6,3,2,5,4]$，则可以得到修改后的收益矩阵为

$$\{c_{ij}-u_j\}=\begin{bmatrix}1&-3&-4&-2&0\\-2&4&-4&0&-5\\0&-1&1&-2&0\\0&-2&2&-1&2\\-4&3&1&-3&0\\-1&-2&0&4&-3\end{bmatrix}$$

这里，所有 $\boldsymbol{u}$ 的和 $\sum\limits_{i\in M}u_i=25$。因此拉格朗日松弛 $\mathrm{IP}(\boldsymbol{u})$ 就可以用观察法求解。例如，对于 $j=2$，如果 $y_2=0$，则我们得到目标函数为 0。如果 $y_2=1$，则在 $x_{22}=1$ 时，由于 $c_{22}-u_2=4$，所以可以使得目标函数增加 4，并且因为 $c_{52}-u_5=3$，相应地可以使得目标函数增加 3，并且 f_jy_j 的值即为 $f_2=4$。所以 $z_2(\boldsymbol{u})=4+3-4=3$。综上，对于设施 2，最优方案就是设置 $y_2=1$，且 $z_2(\boldsymbol{u})=3$。

按照同样的方法，我们依次计算每个设施的情况。最终得到 $\mathrm{IP}(\boldsymbol{u})$ 的最优解为

$$y_2=1,x_{22}=1,x_{52}=1,$$
$$y_4=1,x_{64}=1$$

其余决策变量取值均为 0。最优目标函数值为

$$z(\boldsymbol{u})=3+1+\sum_{i\in M}u_i=29$$

12.4 拉格朗日对偶的加强

上文我们介绍到，如果将复杂约束直接松弛掉，得到的上界（max 问题）太差，但是拉格朗日松弛可以改善这一点，获得一个更紧的上界。接下来我们的问题就是：

（1）通过拉格朗日松弛得到的上界质量有多好？

（2）如何去求解拉格朗日对偶问题？

为了理解拉格朗日对偶问题（LD），可以简单地假设集合 X 中包含大量的有限个点 $\{x_1, x_2, \cdots, x_T\}$。那么有

$$w_{\mathrm{LD}} = \min_{\boldsymbol{u} \geqslant \mathbf{0}} z(\boldsymbol{u}) \tag{12.14}$$

$$= \min_{\boldsymbol{u} \geqslant \mathbf{0}} \left\{ \max_{\boldsymbol{x} \in X} \left[\boldsymbol{c}^{\mathrm{T}} \boldsymbol{x} + \boldsymbol{u}^{\mathrm{T}} (\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}) \right] \right\} \tag{12.15}$$

$$= \min_{\boldsymbol{u} \geqslant \mathbf{0}} \left\{ \max_{t=1,2,\cdots,T} \left[\boldsymbol{c}^{\mathrm{T}} \boldsymbol{x}^t + \boldsymbol{u}^{\mathrm{T}} (\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}^t) \right] \right\} \tag{12.16}$$

式 (12.16) 就等价于

$$\min \quad \eta \tag{12.17}$$

$$\text{s.t.} \quad \eta \geqslant \boldsymbol{c}^{\mathrm{T}} \boldsymbol{x}^t + \boldsymbol{u}^{\mathrm{T}} (\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}^t), \quad \forall t = 1, 2, \cdots, T \tag{12.18}$$

$$\boldsymbol{u} \in \mathbb{R}_+^T, \quad \eta \in \mathbb{R}^1 \tag{12.19}$$

其中，η 是引入的一个变量，是为了表示 $z(\boldsymbol{u})$ 的一个上界。上述涉及 η 的问题是一个线性规划问题。我们用 μ_t 表示约束 (12.18) 的第 t 条约束的对偶变量，则其对偶问题为

$$w_{\mathrm{LD}} = \max \quad \sum_{t=1}^{T} \mu_t \left(\boldsymbol{c}^{\mathrm{T}} \boldsymbol{x}^t \right) \tag{12.20}$$

$$\text{s.t.} \quad \sum_{t=1}^{T} \mu_t \left(\boldsymbol{D}\boldsymbol{x}^t - \boldsymbol{d} \right) \leqslant 0 \tag{12.21}$$

$$\sum_{t=1}^{T} \mu_t = 1 \tag{12.22}$$

$$\boldsymbol{u} \in \mathbb{R}_+^T \tag{12.23}$$

现在，我们令 $\boldsymbol{x} = \sum_{t=1}^{T} \mu_t \boldsymbol{x}^t$，且满足 $\sum_{t=1}^{T} \mu_t = 1, \boldsymbol{u} \in \mathbb{R}_+^T$，我们得到

$$w_{\mathrm{LD}} = \max \quad \boldsymbol{c}^{\mathrm{T}} \boldsymbol{x} \tag{12.24}$$

$$\text{s.t.} \quad \boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d} \tag{12.25}$$

$$\boldsymbol{x} \in \mathbf{Conv}(X) \tag{12.26}$$

注意：上述描述中，$\boldsymbol{x}^t, \forall t = 1, 2, \cdots, T$ 为集合 X 的所有极点，而 $\boldsymbol{x} = \sum_{t=1}^{T} \mu_t \boldsymbol{x}^t$，且满足 $\sum_{t=1}^{T} \mu_t = 1, \boldsymbol{u} \in \mathbb{R}_+^T$，根据凸包的定义，所有 $\boldsymbol{x}$ 组成的集合即为 X 的凸包 $\mathbf{Conv}(X)$。更一般地，我们可以得到，当 X 在任意整数规划 $X = \{\boldsymbol{x} \in \mathbb{Z}_+^n : \boldsymbol{A}\boldsymbol{x} \leqslant \boldsymbol{b}\}$ 的可行域中，上述结果依旧成立。

定理 12.4.1 （Wolsey，1998）$w_{\mathrm{LD}} = \max\{\boldsymbol{c}^{\mathrm{T}} \boldsymbol{x} : \boldsymbol{D}\boldsymbol{x} \leqslant \boldsymbol{d}, \boldsymbol{x} \in \mathbf{Conv}(X)\}$。

这个定理精确地给出了从拉格朗日对偶问题获得的界限有多紧。而在特定的情形下，直接用线性松弛获得的界限并不紧。上述定理提到，我们从拉格朗日对偶问题得到的界限，是

我们考虑约束 $\boldsymbol{Dx} \leqslant \boldsymbol{d}$，且 $\boldsymbol{x}$ 在约束集 $X = \{\boldsymbol{x} \in \mathbb{Z}_+^n : \boldsymbol{Ax} \leqslant \boldsymbol{b}\}$ 的所有整数可行解构成的凸包 $\mathbf{Conv}(X)$ 中的所有可行解的目标函数的最大值。即原问题 IP 的可行域为 $X' = \{\boldsymbol{x} \in \mathbb{Z}_+^n : \boldsymbol{Ax} \leqslant \boldsymbol{b}, \boldsymbol{Dx} \leqslant \boldsymbol{d}\}$，而拉格朗日对偶问题的可行域被扩大为 $\mathbf{Conv}(X) \cap \{\boldsymbol{x} | \boldsymbol{Dx} \leqslant \boldsymbol{d}\}$，且二者的目标函数是一致的，均为 $\max\ \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}$。不难看出，拉格朗日对偶问题得到的上界相比直接删去复杂约束 $\boldsymbol{Dx} \leqslant \boldsymbol{d}$ 得到的上界要提升许多。

定理 12.4.2 （Wolsey，1998）如果有 $X = \{\boldsymbol{x} \in \mathbb{Z}_+^n : \boldsymbol{Ax} \leqslant \boldsymbol{b}\}$ 和凸包 $\mathbf{Conv}(X) = \{\boldsymbol{x} \in \mathbb{R}_+^n : \boldsymbol{Ax} \leqslant \boldsymbol{b}\}$，那么 $w_{\mathrm{LD}} = \max\{\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} : \boldsymbol{Ax} \leqslant \boldsymbol{b}, \boldsymbol{Dx} \leqslant \boldsymbol{d}, \boldsymbol{x} \in \mathbb{R}_+^n\}$。

这是非常有趣的，因为这意味着已经找到了一种求解具有复杂约束的问题的方法，即在求解时并不把这些约束明显地考虑进来，而是将它们加到目标函数中。

上述介绍证明拉格朗日对偶问题已被凸化。即

$$w_{\mathrm{LD}} = \min_{\boldsymbol{u} \geqslant 0} \left\{ \max_{t=1,2,\cdots,T} \left[\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}^t + \boldsymbol{u}^{\mathrm{T}} \left(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}^t \right) \right] \right\} \tag{12.27}$$

因此，拉格朗日对偶问题也可以看作一个寻找分段线性凸函数的最小值的问题，但是该凸函数 $z(u)$ 是不可微的。因为对一个给定的 $t = t_0$，我们是最小化 $t = t_0$ 时的函数

$$w_{\mathrm{LD}}^{t=t_0} = \min_{\boldsymbol{u} \geqslant 0} \left\{ \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}^{t_0} + \boldsymbol{u}^{\mathrm{T}} \left(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}^{t_0} \right) \right\}$$

在这里，$\boldsymbol{D}, \boldsymbol{d}, \boldsymbol{c}, \boldsymbol{x}^{t_0}$ 都是已知量（因为 $\boldsymbol{x}^{t_0}$ 是一个极点），只有 $\boldsymbol{u}$ 是未知量，所以上述函数其实是一个线性函数。针对不同的 t，拉格朗日对偶问题对应不同的线性函数，因此拉格朗日对偶问题是一个分段线性函数，如图 12.2 所示。拉格朗日对偶的目的就是找到使该分段线性函数最小的 $\boldsymbol{u}$。

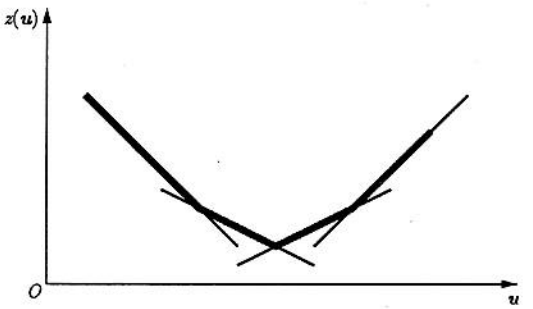


图 12.2　拉格朗日对偶问题的形式

12.5　求解拉格朗日对偶

12.5.1　次梯度算法求解拉格朗日对偶

定理 12.4.1 中给出的线性规划描述了计算 w_{LD} 的方法，该方法需要生成相应的约束（或者割平面），这些约束是用来刻画 $\mathbf{Conv}(X)$ 的。除此之外，还有另外一种可选的方法，那就是次梯度算法（Subgradient Algorithm）。次梯度算法不需要找到凸包，原理更简单且

实现起来更容易。次梯度算法可以用于求解一个分段线性凸函数的最优值。该分段线性凸函数的通用形式如下：

$$\min_{\boldsymbol{u}\geqslant\boldsymbol{0}} \quad f(\boldsymbol{u}) \tag{12.28}$$

$$\text{s.t.} \quad f(\boldsymbol{u}) = \max_{t=1,2,\cdots,T}\left[(\boldsymbol{a}^{\mathrm{T}})^t\boldsymbol{u} - b_t\right] \tag{12.29}$$

在上面介绍的具体的问题中，拉格朗日对偶的形式为

$$w_{\mathrm{LD}} = \min_{\boldsymbol{u}\geqslant\boldsymbol{0}} z(\boldsymbol{u}) \tag{12.30}$$

$$z(\boldsymbol{u}) = \max_{t=1,2,\cdots,T}\left[\boldsymbol{c}^{\mathrm{T}}\boldsymbol{x}^t + \boldsymbol{u}^{\mathrm{T}}\left(\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}^t\right)\right] \tag{12.31}$$

二者的形式是相同的。

次梯度（Subgradient）就是梯度（Gradient）的一种广义化。梯度就是导数，是函数值差值除以自变量差值取极限。但是拉格朗日对偶不可微。因此我们用相邻两次的变化率来代替梯度，这种计算方法类似于梯度的计算方法，计算出来的变化率叫作次梯度。次梯度算法有很多改进版本，这里仅介绍最基本的版本。

定义 12.5.1 （Wolsey，1998）*凸函数* $f:\mathbb{R}^m\to\mathbb{R}^1$ *关于* $\boldsymbol{u}$ *的次梯度是一个向量* $\boldsymbol{\gamma}(\boldsymbol{u})\in\mathbb{R}^m$，*满足* $f(\boldsymbol{v})\geqslant f(\boldsymbol{u})+\boldsymbol{\gamma}(\boldsymbol{u})^{\mathrm{T}}(\boldsymbol{v}-\boldsymbol{u}),\forall\boldsymbol{v}\in\mathbb{R}^m$。

对于一个连续可微的凸函数 f，则 $\boldsymbol{\gamma}(\boldsymbol{u})=\nabla f(\boldsymbol{u})=\left[\dfrac{\partial f}{\partial u_1},\cdots,\dfrac{\partial f}{\partial u_m}\right]^{\mathrm{T}}$ 是 f 在 $\boldsymbol{u}$ 处的梯度。

下面我们给出次梯度算法的较为简略的伪代码。

Algorithm 8 拉格朗日对偶问题的次梯度算法

初始化：设置拉格朗日乘子的初始值 $\boldsymbol{u}\leftarrow\boldsymbol{u}^0$
$k\leftarrow 0$
while $k<$ maxIter **do**
  $\boldsymbol{u}\leftarrow\boldsymbol{u}^k$
  /* IP($\boldsymbol{u}^k$) 带有整数约束 */
  求解拉格朗日松弛问题 IP($\boldsymbol{u}^k$)，其解为 $\boldsymbol{x}(\boldsymbol{u}^k)$
  (****** 计算次梯度和步长 ******)
  用 $\boldsymbol{x}(\boldsymbol{u}^k)$ 计算次梯度：$\boldsymbol{\gamma}^k\leftarrow[\boldsymbol{d}-\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}^k)]$
  步长 $\mu^k\leftarrow$ 任意步长函数
  更新拉格朗日乘子：$\boldsymbol{u}^{k+1}\leftarrow\max\{\boldsymbol{u}^k-\mu^k(\boldsymbol{d}-\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}^k)),\ \boldsymbol{0}\}$
  $k\leftarrow k+1$
end while

注意：*在上述伪代码中，向量* $\boldsymbol{d}-\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}^k)$ *是函数* $z(\boldsymbol{u})$ *在* $\boldsymbol{u}^k$ *的次梯度。*

在每一次迭代中，我们都是从当前点 $\boldsymbol{u}^k$ 出发，逆着次梯度的方向，一步一步更新，也就是每一步的更新量为 $-\mu^k(\boldsymbol{d}-\boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}^k))$，其中，$\mu^k$ 是第 k 步迭代的步长。上述伪代码

是非常简洁明了的，只是给出了大致的思路，但是对于一些具体细节，如怎样设置更新步长 μ^k 等，并没有给出详细的阐述。而如何设置每一步的步长 $\{\mu^k\}_{k=1}^{\infty}$，对于次梯度算法是非常重要的。

下面介绍有关步长设置的内容。在这之前，首先来看下面的定理。

定理 12.5.1 （Wolsey，1998）（1）如果 $\sum_k \mu^k \to \infty$，且当 $k \to \infty$，$\mu^k \to 0$，则 $z(\boldsymbol{u}^k) \to w_{\mathrm{LD}}$，其中，$w_{\mathrm{LD}}$ 是拉格朗日对偶（LD）的最优值。

（2）如果对于某个参数 $\rho < 1$，有 $\mu^k = \mu^0 \rho^k$，当 μ^0 和 ρ 足够大时，则有 $z(\boldsymbol{u}^k) \to w_{\mathrm{LD}}$。

（3）如果 $\bar{w} \geqslant w_{\mathrm{LD}}$，且 $\mu^k = \dfrac{\epsilon_k \left[z\left(\boldsymbol{u}^k\right) - \bar{w}\right]}{\|\boldsymbol{d} - \boldsymbol{D}\boldsymbol{x}\left(\boldsymbol{u}^k\right)\|^2}$，且 $0 < \epsilon_k < 2$，则 $z(\boldsymbol{u}^k) \to \bar{w}$。或者算法将会找到对于有限的 k 及对应的 $\boldsymbol{u}^k$，满足 $\bar{w} \geqslant z(\boldsymbol{u}^k) \geqslant w_{\mathrm{LD}}$。

上述规则 (1) 中的步长设置思路保证了算法可以收敛，但是由于序列 $\{\mu^k\}$ 必须是发散的，例如 $\mu^k = 1/k$。算法收敛非常缓慢，不太实用。

另一方面，第 (2) 和 (3) 条规则设置步长的思路收敛速度就快很多，但是每次计算新的更新步长可能不是很方便。注意，$\|\boldsymbol{b} - \boldsymbol{D}\boldsymbol{x}(\boldsymbol{u}^k)\|^2$ 是对其所有元素的平方加和。

使用规则 (2)，则 μ^0 和 ρ 的初始值就需要足够大。否则几何级数 $\mu^0\rho^k$ 很快就会趋近于 0，而且很可能 $\boldsymbol{u}^k$ 在没有得到最优值点之前就已经收敛了。在实际中，我们并不是每次都去减小 μ^k，而是在好几次迭代之后，再去减小 μ^k。为此，我们可以设置一个参数 λ，每迭代 λ 次，我们更新一次 μ^k。

如果用规则 (3)，一个困难就是一个对偶上界 $\bar{w} \geqslant w_{\mathrm{LD}}$ 一般是不知道的。在实际中很有可能我们知道一个原问题的比较好的下界 $\underline{w} \leqslant w_{\mathrm{LD}}$。因此，这样的下界 $\underline{w}$ 在初始阶段就被用来代替 $\bar{w}$。但是，如果出现 $\underline{w} < w_{\mathrm{LD}}$，且式 $z(\boldsymbol{u}^k) - \underline{w}$ 在更新的过程中，μ_k 始终不趋向于 0，因而序列 $\{\mu^k\}$ 和 $\{z(\boldsymbol{u}^k)\}$ 也就无法收敛。出现上述情况时，我们必须增大 $\underline{w}$ 的值。

上述内容提供了设置步长 μ^k 的具体方法，为我们实现次梯度算法求解拉格朗日对偶提供了极大帮助。另外，文献（Kalvelagen，2002）提供了一种更为详细的次梯度算法的伪代码，其中 θ_0 为参数。

Algorithm 9 拉格朗日对偶问题的次梯度算法

输入：初始上界 UB，拉格朗日乘子初始值 $\boldsymbol{u}_0 \geqslant \boldsymbol{0}$
初始化：$\theta_0 \leftarrow 2$
(**** 次梯度迭代 ****)
for $k <$ `maxIter` **do**
  $\boldsymbol{\gamma}^k \leftarrow g(\boldsymbol{x}^k)$ (γ^k 就是 $z(\boldsymbol{u}^k)$ 的次梯度，一般用对应约束的松弛变量作为次梯度)
  $t_k \leftarrow \dfrac{\theta_k \left[\mathrm{UB} - z\left(\boldsymbol{u}^k\right)\right]}{\|\boldsymbol{\gamma}^k\|^2}$ (步长)
  $u^{k+1} \leftarrow \max\{0, \boldsymbol{u}^k + t_k\boldsymbol{\gamma}^k\}$
  if $\|\boldsymbol{u}^{k+1} - \boldsymbol{u}^k\| < \epsilon$ **then** /*$\|\boldsymbol{u}^{k+1} - \boldsymbol{u}^k\|$ 是对向量的所有分量加和 */
    Stop
  end if

```
    if 超过 K 步迭代还没有改进 then
        θ_{k+1} ← θ_k / 2
    else
        θ_{k+1} ← θ_k
    end if
    k ← k + 1
end for
```

12.5.2 应用案例：拉格朗日松弛求解 TSP

本节介绍如何使用拉格朗日松弛求解对称旅行商问题（Symmetric Traveling Salesman Problem，STSP）。

STSP 的标准模型见 2.8 节。下面将每个节点的度均为 2 的约束松弛掉，即下面的约束：

$$\sum_{e\in\delta(i)} x_e = 2, \quad \forall i \in V$$

因此，拉格朗日乘子 u_i^k 的更新方程为

$$u_i^{k+1} = u_i^k + \mu^k \left(2 - \sum_{e\in\delta(i)} x_e\left(\boldsymbol{u}^k\right)\right)$$

这里，u_i^{k+1} 表示第 $k+1$ 次迭代时次梯度中第 i 个维度的取值。这里，步长更新如果用第 (3) 种规则，则步长 μ_k 即为

$$\mu_k = \frac{\epsilon_k\left[\bar{w} - z\left(\boldsymbol{u}^k\right)\right]}{\sum\limits_{i\in V}\left(2 - \sum\limits_{e\in\delta(i)} x_e\left(\boldsymbol{u}^k\right)\right)^2}$$

上式中由于 STSP 是一个 min 问题，因此其拉格朗日对偶也是一个 min 问题，所以分子中用 $\bar{w} - z\left(\boldsymbol{u}^k\right)$。

下面以一个非常具体的算例来解释拉格朗日松弛求解 STSP 的完整过程（Wolsey，1998）。令距离矩阵为

$$\{c_e\} = \begin{bmatrix} - & 30 & 26 & 50 & 40 \\ - & - & 24 & 40 & 50 \\ - & - & - & 24 & 26 \\ - & - & - & - & 30 \\ - & - & - & - & - \end{bmatrix}$$

假设已经用启发式算法找到了一个环路 $(1,2,3,4,1)$，总距离为 148。但是并没有任何下界。用规则 (3) 更新对偶变量 $\boldsymbol{u}$。并且取 $\epsilon = 1$，由于没有可用的下界，就只能用 $\bar{w} = 148$，这点在前文中也有相应的介绍。

第 1 次迭代：设置 $\boldsymbol{u}^1=[0,0,0,0,0]^{\mathrm{T}}$。修正的成本矩阵为 $\bar{c}^1=c$。下面给出一个最优的 1-tree，如图 12.3所示。并且 $z(\boldsymbol{u}^1)=130\leqslant z$，由于 $\mathbf{2}-\sum\limits_{e\in\delta(i)}x_e\left(\boldsymbol{u}^k\right)=(0,0,-2,1,1)$。我们有

$$\boldsymbol{u}^2=\boldsymbol{u}^1+\left[\frac{148-130}{6}\right][0,0,-2,1,1]^{\mathrm{T}}$$

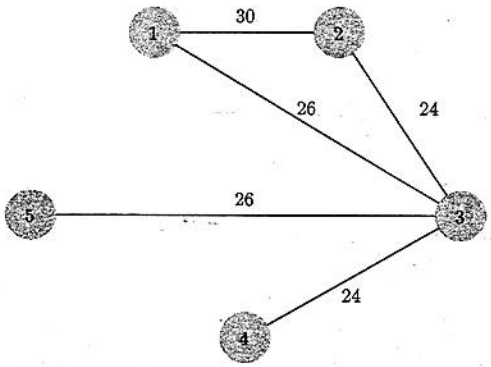


图 12.3　$\bar{c}^1$ 对应的最优 1-tree

第 2 次迭代：设置 $\boldsymbol{u}^2=[0,0,-6,3,3]^{\mathrm{T}}$。新的修正的成本矩阵为

$$\left\{\bar{c}_e^2\right\}=\begin{bmatrix}- & 30 & 32 & 47 & 37\\ - & - & 30 & 37 & 47\\ - & - & - & 27 & 29\\ - & - & - & - & 24\\ - & - & - & - & -\end{bmatrix}$$

得到 $z(\boldsymbol{u}^2)=143+\sum\limits_i u_i^2=143$，并且

$$\boldsymbol{u}^3=\boldsymbol{u}^2+\left[\frac{148-143}{2}\right][0,0,-1,0,1]^{\mathrm{T}}$$

新的最优 1-tree 如图 12.4 所示。

第 3 次迭代：设置 $\boldsymbol{u}^3=\left[0,0,-\frac{17}{2},3,\frac{11}{2}\right]^{\mathrm{T}}$。新的修正的成本矩阵为

$$\left\{\bar{c}_e^3\right\}=\begin{bmatrix}- & 30 & 34.5 & 47 & 34.5\\ - & - & 32.5 & 37 & 44.5\\ - & - & - & 29.5 & 29\\ - & - & - & - & 21.5\\ - & - & - & - & -\end{bmatrix}$$

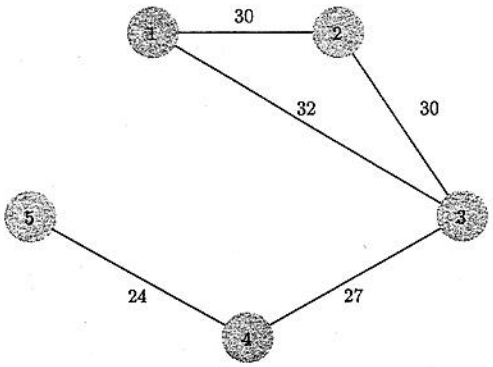


图 12.4 $\bar{c}^2$ 对应的最优 1-tree

新的 1-tree 如图 12.5 所示，我们得到了一个下界 $z(\boldsymbol{u}^3)=147.5$。由于参数 c 都是整数的，所以可知，原模型的最优值 z 一定是整数，因此 $z \geqslant \lceil 147.5 \rceil = 148$。由于之前已经知道了一个成本为 148 的整数解，综合以上信息，易得，成本为 148 的这条路径就是最优路径。

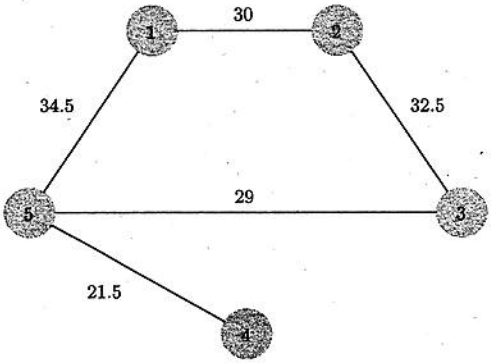


图 12.5 $\bar{c}^3$ 对应的最优 1-tree

由于次梯度算法常常在拉格朗日对偶的目标值 w_{LD} 到达最优之前就结束了，并且由于在很多时候也会存在一个对偶 Gap ($w_{\mathrm{LD}} > z$)，因此拉格朗日松弛经常被嵌入分支定界算法中使用。

12.6 如何选择拉格朗日松弛

假设需要求解的问题是下面的形式：

$$
\begin{aligned}
(\mathrm{IP}) \qquad \max \quad & z = \boldsymbol{c}^{\mathrm{T}}\boldsymbol{x} && (12.32)\\
\text{s.t.} \quad & \boldsymbol{A}^1\boldsymbol{x} \leqslant \boldsymbol{b}^1 && (12.33)\\
& \boldsymbol{A}^2\boldsymbol{x} \leqslant \boldsymbol{b}^2 && (12.34)
\end{aligned}
$$

$$x \in \mathbb{Z}_+^n \tag{12.35}$$

如果想用拉格朗日松弛来处理这个问题，就需要做一个决定，是将两组约束都松弛，还是只松弛一组？如果只松弛一组，那么选择哪一组进行松弛？我们必须要在下面的几个指标中做一个权衡：

（1）产生的拉格朗日对偶的 Bound w_{LD} 的强度；

（2）求解拉格朗日松弛问题 $\mathrm{IP}(\boldsymbol{u})$ 的难易程度；

（3）求解产生的拉格朗日对偶：$w_{\mathrm{LD}} = \min\limits_{\boldsymbol{u}\geqslant 0} z(\boldsymbol{u})$ 的难易程度。

对于第（1）点，定理 12.4.1已经给出了答案。

对于第（2）点，求解 $\mathrm{IP}(\boldsymbol{u})$ 的难易程度需要根据具体问题具体分析。但是，如果我们知道 $\mathrm{IP}(\boldsymbol{u})$ 是非常容易的，比如 $\mathrm{IP}(\boldsymbol{u})$ 变成了一个线性规划。

对于第（3）点，用次梯度算法或者其他算法无法提前预估该问题的难易程度。但是可以根据对偶变量的个数做一个估计。

为了说明上面那些权衡，这里以一个非常具体的问题加以说明。以广义指派问题（Generalized Assignment Problem，GAP）为例，首先，其数学模型如下：

$$\max z = \sum_{j=1}^{n}\sum_{i=1}^{m} c_{ij}x_{ij} \tag{12.36}$$

$$\sum_{j=1}^{n} x_{ij} \leqslant 1, \qquad \forall i = 1,2,\cdots,m \tag{12.37}$$

$$\sum_{i=1}^{m} a_{ij}x_{ij} \leqslant b_j, \qquad \forall j = 1,2,\cdots,n \tag{12.38}$$

$$\boldsymbol{x} \in \{0,1\}^{m\times n} \tag{12.39}$$

这里考虑 3 种拉格朗日松弛。在第 1 种情况下，将两组约束 (12.37) 和 (12.38) 都对偶到目标函数中，有 $w_{\mathrm{LD}}^1 = \min\limits_{\boldsymbol{u}\geqslant 0,\boldsymbol{v}\geqslant 0} w^1(\boldsymbol{u},\boldsymbol{v})$，其中，

$$w^1(\boldsymbol{u},\boldsymbol{v}) = \max_{\boldsymbol{x}} \sum_{j=1}^{n}\sum_{i=1}^{m} (c_{ij} - u_i - a_{ij}v_j)x_{ij} + \sum_{i=1}^{m} u_i + \sum_{j=1}^{n} v_j b_j$$

$$\boldsymbol{x} \in \{0,1\}^{m\times n}$$

如果只选择对偶指派的相关约束 (12.37)，则会得到 $w_{\mathrm{LD}}^2 = \min\limits_{\boldsymbol{u}\geqslant 0} w^2(\boldsymbol{u})$，其中，

$$w^2(\boldsymbol{u}) = \max_{\boldsymbol{x}} \sum_{j=1}^{n}\sum_{i=1}^{m} (c_{ij} - u_i)x_{ij} + \sum_{i=1}^{m} u_i$$

$$\text{s.t.} \sum_{i=1}^{m} a_{ij}x_{ij} \leqslant b_j, \quad \forall j = 1,2,\cdots,n$$

$$\boldsymbol{x} \in \{0,1\}^{m\times n}$$

如果选择对偶背包约束 (12.38)，则得到 $w_{\mathrm{LD}}^3 = \min\limits_{v \geqslant 0} w^3(v)$，其中，

$$w^3(u) = \max_{x} \sum_{j=1}^{n} \sum_{i=1}^{m} (c_{ij} - a_{ij}v_j) x_{ij} + \sum_{j=1}^{n} v_j b_j$$

$$\text{s.t.} \sum_{j=1}^{n} x_{ij} \leqslant 1, \quad \forall i = 1, 2, \cdots, m$$

$$x \in \{0,1\}^{m \times n}$$

基于定理 12.4.1，可得 $w_{\mathrm{LD}}^1 = w_{\mathrm{LD}}^3 = z_{\mathrm{LP}}$，由于对任意的 i

$$\mathbf{Conv}\left\{x : \sum_{j=1}^{n} x_{ij} \leqslant 1, x_{ij} \in \{0,1\}, \forall j = 1, 2, \cdots, n\right\}$$

$$= \left\{x : \sum_{j=1}^{n} x_{ij} \leqslant 1, 0 \leqslant x_{ij} \leqslant 1, \forall j = 1, 2, \cdots, n\right\}$$

计算 $w^1(u, v)$ 和 $w^3(v)$ 都可以使用观察法。对于 $w^1(u, v)$，我们发现该问题可以按照变量分解，而 $w^3(v)$ 是根据 $\forall j = 1, 2, \cdots, n$ 来分解的。就求解拉格朗日对偶问题而言，w_{LD}^3 比 w_{LD}^1 更容易，因为 w_{LD}^3 只有 m 个对偶变量，但是 w_{LD}^1 却有 $m+n$ 个对偶变量。

第二个松弛其实会提供一个更紧的界限 $w_{\mathrm{LD}}^2 \leqslant z_{\mathrm{LP}}$，由于对于特定的 j 来讲，有

$$\mathbf{Conv}\left\{x : \sum_{i=1}^{m} a_{ij} x_{ij} \leqslant b_j, x_{ij} \in \{0,1\}^m\right\}$$

$$\subseteq \left\{x : \sum_{i=1}^{m} a_{ij} x_{ij} \leqslant b_j, 0 \leqslant x_{ij} \leqslant 1, \forall i = 1, 2, \cdots, m\right\}$$

然而，这样做会使得拉格朗日子问题包含 m 个 0-1 背包问题。

12.7 Python 调用 Gurobi 实现拉格朗日松弛求解选址–运输问题

本章将详细介绍用拉格朗日松弛求解选址–运输问题（Location Transport Problem，LTP）的模型重构、伪代码及其 Python 实现。本部分主要内容参考自文献 Kalvelagen，2002。

12.7.1 拉格朗日松弛应用案例：选址–运输问题

本节以选址–运输问题（LTP）为例来介绍拉格朗日松弛算法的应用。回顾第 2 章中对该问题的介绍，引入下面两个决策变量：

（1）x_{ij}：从配送中心 $i \in D$ 到客户点 $j \in C$ 的配送量；

（2）y_i：配送中心 i 是否被选，如果被选则为 1，否则为 0。

基于此，LTP 可以建模为下面的混合整数规划模型：

$$\min \quad \sum_{i\in D}\sum_{j\in C} c_{ij}x_{ij} \tag{12.40}$$

$$\text{s.t.} \quad \sum_{j\in C} x_{ij} \leqslant s_i y_i, \qquad \forall i \in D \tag{12.41}$$

$$\sum_{i\in D} x_{ij} \geqslant d_j, \qquad \forall j \in C \tag{12.42}$$

$$\sum_{i\in D} y_i \leqslant P \tag{12.43}$$

$$x_{ij} \geqslant 0\text{且为整数}, y_i \in \{0,1\}, \quad \forall i \in D, \forall j \in C \tag{12.44}$$

将约束 (12.42) 松弛，则可以得到拉格朗日松弛函数

$$L(\mu) = \sum_{i\in D}\sum_{j\in C} c_{ij}x_{ij} + \sum_{j\in C} u_j \left(d_j - \sum_{i\in D} x_{ij} \right) \tag{12.45}$$

$$= \sum_{i\in D}\sum_{j\in C} c_{ij}x_{ij} + \sum_{j\in C} u_j d_j - \sum_{i\in D}\sum_{j\in C} u_j x_{ij} \tag{12.46}$$

相应地，拉格朗日对偶问题即为

$$\min \quad \sum_{i\in D}\sum_{j\in C} c_{ij}x_{ij} + \sum_{j\in C} u_j d_j - \sum_{i\in D}\sum_{j\in C} u_j x_{ij} \tag{12.47}$$

$$\text{s.t.} \quad \sum_{j\in C} x_{ij} \leqslant s_i y_i, \qquad \forall i \in D \tag{12.48}$$

$$\sum_{i\in D} y_i \leqslant P \tag{12.49}$$

$$x_{ij} \geqslant 0\text{且为整数}, y_i \in \{0,1\}, \qquad \forall i \in D, \forall j \in C \tag{12.50}$$

其中，拉格朗日乘子（或者说对偶变量）u_j 需要一个初始值，然后再在算法迭代中根据次梯度更新。而对偶变量 u_j 的次梯度可以用原问题约束 (12.42) 的松弛变量（$\leqslant$ 约束）或者剩余变量（$\geqslant$ 约束）来衡量，本例中对应的是剩余变量。即

$$u_j = \mathtt{slack}_j \tag{12.51}$$

$$= \sum_{i\in D} x_{ij} - d_j \tag{12.52}$$

由于在模型 (12.47) 中将约束 $\sum_{i\in D} x_{ij} \geqslant d_j, \forall j \in C$ 松弛了，因此直接从模型得到约束 (12.42) 的松弛变量是不能实现的。因此需要求解模型 (12.47)，得到其解 $\bar{x}_{ij}$，然后根据 $\bar{x}_{ij}$ 计算得到剩余变量 $\mathtt{slack}_j$。

得到剩余变量 $\mathtt{slack}_j$ 之后，还需要确定用来更新拉格朗日乘子 u_j 的步长 t_j。这里采用下面的方法来设置步长（k 表示第 k 步迭代）：

$$t_j^k = \frac{\theta_k \left[\mathrm{UB} - z\left(\boldsymbol{u}\right)\right]}{\|\gamma\|^2} = \frac{\theta_k \left[\mathrm{UB} - z\left(\boldsymbol{u}\right)\right]}{\sum\limits_{j \in C} \mathtt{slack}_j^2} \tag{12.53}$$

其中，θ 是一个标量，也是随着迭代更新的。UB 是当前最好的上界，$z(\boldsymbol{u})$ 是当前目标值。这些都准备好之后，用下面的式子来更新拉格朗日乘子：

$$u_j^{k+1} = \max\left\{0,\ u_j^k + t_j^k \mathtt{slack}_j\right\} \tag{12.54}$$

> 由于 u_j 是拉格朗日乘子，必须满足非负条件，也就是 $u_j \geqslant 0$，因此我们让 u_j 取大括号中二者最大值，避免 u_j 取负值。

因此，用次梯度算法求解 LTP 的拉格朗日对偶问题的伪代码如下。

Algorithm 10 拉格朗日对偶问题的次梯度算法

Input: 初始化拉格朗日乘子 $\boldsymbol{u}_0 \leftarrow \mathbf{0}$

Input: 初始化上界 $\mathrm{UB} \leftarrow \sum\limits_{i \in D} c_i = \sum\limits_{i \in D} \max\limits_{j} \{c_{ij}\}$

Output: $z(\boldsymbol{u}^*)$ 和拉格朗日乘子 $\boldsymbol{u}^*$

初始化：$\theta_0 \leftarrow 2$

次梯度迭代

for $k <$ `maxIter` **do**

  $\bar{x}_{ij}, z(\boldsymbol{u}^k) \leftarrow$ 求解松弛模型 (12.47)

  计算次梯度：$\gamma_j^k \leftarrow \sum\limits_{i \in D} \bar{x}_{ij} - d_j$（$\gamma_j^k$ 是 $z(\boldsymbol{u}^k)$ 的次梯度）

  （一般用对应约束的松弛变量作为次梯度）

  更新步长：$t_k \leftarrow \dfrac{\theta_k \left[\mathrm{UB} - z\left(\boldsymbol{u}^k\right)\right]}{\sum\limits_{j \in C} \left(\gamma_j^k\right)^2}$

  $\boldsymbol{u}^{k+1} \leftarrow \max\{0, \boldsymbol{u}^k + t_k \boldsymbol{\gamma}^k\}$

  if $\|\boldsymbol{u}^{k+1} - \boldsymbol{u}^k\| < \epsilon$ **then**

    Stop

  end if

  if 超过 K 步迭代还没有任何改进 **then**

    $\theta_{k+1} \leftarrow \dfrac{\theta_k}{2}$

  else

    $\theta_{k+1} \leftarrow \theta_k$

  end if

  $k \leftarrow k + 1$

18: **end for**

下面用 Python 调用 Gurobi 实现上述算法。

12.7.2 Python 代码实现

注意，在算例文件 location_transport_instance.txt 中：第 1 行为可选设施的数量上限；第 2 行为客户个数；第 3 行为 30 个候选设施 (配送中心或者仓库) 的供应量；第 4 行为 30 个客户点的需求量；第 5~34 行为配送费用矩阵 $c_{ij}, \forall i \in D, \forall j \in C$。

算例测试代码如下：

test instance

```
data = Data()
data = readData(data, 'location_transport_instance.txt')

# initialize parameters
maxIter = 200
noChangeCntLimit = 5
stepSizeLog = []
thetaLog = []
LBlog = []
UBlog = []

var_x = []
var_y = []
relaxedCons = []    # relaxed constraints

LTP_model, var_x, var_y, relaxedCons = creatModel(data, var_x, var_y,
       relaxedCons)

subGradientSolve(data, LTP_model, var_x, var_y, relaxedCons, maxIter,
       noChangeCntLimit, LBlog, UBlog,thetaLog, stepSizeLog)
```

运行结果如下。

test instance

```
[Out]:

   --------------  Iteration log information --------------

  Iter        LB              UB          theta        stepSize
  ..............................................................
    8     1084.122883     2684.000000    1.000000    1.259523
```

```
 9     1084.122883     2684.000000     1.000000     0.679221
10     1093.244186     2684.000000     1.000000     2.367196
11     1093.244186     2684.000000     1.000000     1.279174
12     1093.244186     2684.000000     1.000000     0.622132
13     1258.238295     2684.000000     1.000000     1.087538
.....................................................................
20     1317.915751     2684.000000     0.500000     1.112446
21     1317.915751     2684.000000     0.500000     1.062753
22     1317.915751     2684.000000     0.500000     0.296870
23     1465.210699     1592.000000     0.500000     2.199981
24     1465.210699     1592.000000     0.500000     0.191505
25     1465.210699     1592.000000     0.500000     0.150506
.....................................................................
177    1547.479347     1592.000000     0.000244     0.000111
178    1547.479347     1592.000000     0.000244     0.000069
179    1547.479347     1592.000000     0.000244     0.000061
180    1547.479347     1592.000000     0.000244     0.000101
181    1547.479347     1592.000000     0.000122     0.000049
182    1547.480336     1592.000000     0.000122     0.000062
.....................................................................
193    1547.481684     1592.000000     0.000122     0.000051
194    1547.481684     1592.000000     0.000122     0.000049
195    1547.481941     1592.000000     0.000122     0.000020
196    1547.482079     1592.000000     0.000122     0.000057
197    1547.482079     1592.000000     0.000122     0.000055
198    1547.482171     1592.000000     0.000122     0.000042
199    1547.482171     1592.000000     0.000122     0.000019
```

完整代码见附配资源。

第13章　列生成算法

列生成算法（Column Generation Algorithm）是混合整数规划中一个非常强大的精确算法。该算法由 Gilmore 和 Gomory 于 1961 年在研究下料问题（Cutting Stock Problem）的文章中首次提出（Gilmore and Gomory，1961），其基本原理与单纯形法迭代过程中选择入基变量的原理基本相同。列生成算法也经常与之前章节介绍过的分支定界算法，还有之后章节将要介绍的 DW 分解（Dantzig-Wolfe Decomposition）算法组合使用，即所谓的分支定价（Branch and Price）算法。本章我们就来详细介绍列生成算法。

13.1　为什么用列生成算法

下面考虑这个只有 2 个决策变量的简单线性规划问题：

$$\begin{aligned} \max\ z =& 7x_1+5x_2 \\ \text{s.t.}\quad & x_1+ x_2 \leqslant 5 \\ & 7x_1+3x_2 \leqslant 21 \\ & x_1,\quad x_2 \geqslant 0 \end{aligned} \tag{13.1}$$

使用 Python 调用 Gurobi 建模，代码如下：

simple LP

```
from gurobipy import *
import pandas as pd
import numpy as np

model = Model('LP')
X = {}
for i in range(2):
    X[i+1] = model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
        'x' + str(i+1))

model.setObjective(7 * X[1] + 5 * X[2], GRB.MAXIMIZE)
model.addConstr(X[1] + X[2] <= 5)
model.addConstr(7 * X[1] + 3 * X[2] <= 21)
model.optimize()
```

```
print('Objective = \t', model.ObjVal)
for var in model.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =      28.0
x1 =     1.5
x2 =     3.5
```

由最优解可知，约束个数为 2，最终 2 个决策变量都大于 0。这是约束个数等于变量个数的情况。但是在现实世界应用场景中，约束个数和变量个数往往不相等。假设考虑下面这个变量个数远大于约束个数的线性规划：

$$\begin{aligned} \max\ z = & 7x_1+5x_2-x_3-x_4-x_5-x_6 \\ \text{s.t.}\quad & x_1+x_2+x_3+x_5 \leqslant 5 \\ & 7x_1+3x_2+x_4+x_6 \leqslant 21 \\ & x_1,\ x_2,\ x_3,\ x_4,\ x_5,\ x_6 \geqslant 0 \end{aligned} \tag{13.2}$$

Python 代码如下：

simple LP2

```
from gurobipy import *
import pandas as pd
import numpy as np

model = Model('LP2')
X = {}
for i in range(6):
    X[i+1] = model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name=
        'x' + str(i+1))

model.setObjective(7 * X[1] + 5 * X[2] - X[3] - X[4] - X[5] - X[6], GRB.
    MAXIMIZE)
model.addConstr(X[1] + X[2] + X[3] + X[5] <= 5)
model.addConstr(7 * X[1] + 3 * X[2] + X[4] + X[6] <= 21)
model.optimize()
print('Objective = \t', model.ObjVal)
for var in model.getVars():
    print(var.varName, '= \t', var.x)

[Out]:
Objective =      28.0
```

```
x1 =      1.5
x2 =      3.5
x3 =      0.0
x4 =      0.0
x5 =      0.0
x6 =      0.0
```

从结果可得，$x_1 = 1.5, x_2 = 3.5$，但是其余变量取值均为零，并且该解与 (13.1) 的最优解完全相同。我们容易得出，约束矩阵为

$$\boldsymbol{A} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 \\ 7 & 3 & 0 & 1 & 0 & 1 \end{bmatrix}$$

该矩阵的秩为 2，因此在单纯形法的迭代中，每一步迭代的基变量集合中仅有 2 个变量。其余 4 个均为非基变量。另外，该问题为 max 问题，目标函数中，变量 x_3, x_4, x_5, x_6 的系数为 -1，因此这几个变量在最终的最优单纯形表中一定都不是基变量，也就是说，这几个变量是冗余的，根本不参与实际的有效的计算。这样的决策变量以及决策变量对应的列，可以直接将其从模型中删除，并且删除之后不影响整个问题的最优解。就像上面的例子，线性规划 (13.2) 删除了变量 x_3, x_4, x_5, x_6 之后，就变成模型 (13.1)，二者的最优解是相同的，并且模型 (13.1) 更简洁，求解起来更容易。事实上，模型 (13.1) 中的两个变量 x_1, x_2 就是模型 (13.2) 的最优基。

由此可以联想到，在一些实际问题中，也许没有必要将所有的决策变量都加入最终模型中，而只是添加那些在单纯形法迭代中一定会入基或者有很大潜力会入基的变量，从而使被加入的变量（也就是列）真正参与到改进目标函数的过程中。在整个过程中，我们将忽略那些在单纯形法的迭代中不会入基的变量。按照这种操作获得的模型就更轻量，求解也更快。换句话说，我们可以通过加入一部分很有潜力的变量组成新的模型，这些变量的集合包含了原来问题的最优基（也就是包含了原问题最优解中的所有基变量），这个新的模型就等价于原来的模型，而且更容易求解。我们本章要介绍的列生成算法，正是这样一种强大的算法。有别于我们一般熟知的按行建模的方法（变量固定，根据变量构建约束），列生成算法是一种按列建模的方法，变量数量（模型的列数）刚开始是未知的，需要通过一个定价子问题（Pricing Problem）去动态地生成，然后添加到主问题（Master Problem）中去。这种建模方法也被广泛地应用到 TSP、VRP、MCNF 等问题中。本章就来详细地介绍这种方法。

13.2 下料问题

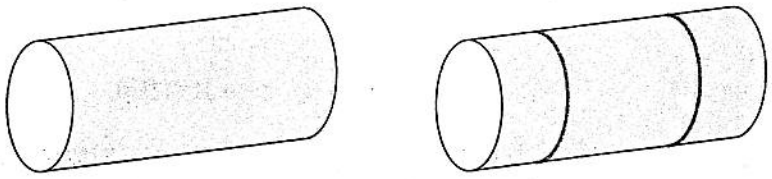
13.2.1 引例

讲到列生成算法，就不得不提及一个列生成算法的经典案例：下料问题（Cutting Stock Problem）。实际上，列生成算法就是 Gilmore 和 Gomory 在研究下料问题时提出来的

(Gilmore and Gomory，1961)。该问题经常作为列生成算法的入门例子。下料问题的描述如下。

这里考虑某工厂生产一批金属棒材，原材料均为等长的圆柱形金属，而客户需要的棒材长度大小不一，数量相异。作为该工厂管理人员，我们应该如何去制定棒材的切割策略，才能最大限度地减少原料棒材的浪费，或者切割尽量少的原材料棒材去满足客户的需求？

为了方便读者理解，下面给出如图 13.1 所示的棒材切割示意图。



(a) 原料棒材　　(b) 切割方案例子

图 13.1　棒材和切割方案

下面以一个具体的算例来作为引入，如下。

> 一家棒料销售公司销售 10 英寸、11 英寸和 19 英寸的棒料产品。客户需要 46 个 10 英寸、22 个 11 英寸和 43 个 19 英寸的产品。棒料销售公司需要截断 80 英寸的原材料棒料来满足客户的需求，同时最小化使用棒材的根数。问该公司最少需要多少棒材，以及如何切割棒材？

每一条长为 80 英寸的棒材都有若干种切割成所需棒材的方法，例如一根棒材可以刚好切成 8 个 10 英寸的棒料；也可以切成 4 个 10 英寸的和 2 个 19 英寸的棒材，剩余 2 英寸废料……还有很多其他切割方案。

这里给出一些很容易想到的切割方案，如表 13.1 所示。表中给出了 5 种待选的切割方案，但是这仅仅是所有可能的切割方案中的一小部分。

表 13.1　一些候选切割方案

	数　量			浪费的量	切割的 80 英寸的棒料的数量
	10 英寸	11 英寸	19 英寸		
Pattern1	8	0	0	0	x_1
Pattern2	6	1	0	9	x_2
Pattern3	6	0	1	1	x_3
Pattern4	5	1	1	0	x_4
Pattern5	5	2	0	8	x_5
总需求	**46**	**22**	**43**		

对于规模较大的问题，很难穷举所有的切割方案。假如棒材足够长，所需棒材长度的

种类也比较多时，切割种类就呈指数级增长。此时如果试图穷举所有方案，那么，仅仅穷举方案的时间也许都会比求解问题本身的时间更长。所以穷举的方法是不可行的。

回到上面的例子，针对表 13.1 中列出的 5 种切割方案给出下料问题的模型：

$$
\begin{aligned}
\min\ z = {} & x_1 + x_2 + x_3 + x_4 + x_5 \\
\text{s.t.}\quad & 8x_1+6x_2+6x_3+5x_4+5x_5 \geqslant 46 \\
& x_2 + x_4+2x_5 \geqslant 22 \\
& x_3+ x_4 \geqslant 43 \\
& x_1,\ x_2,\ x_3,\ x_4,\ x_5 \geqslant 0 \text{ 且为整数}
\end{aligned}
\tag{13.3}
$$

上述模型中，每列都对应一种切割方案。由于我们并没有把所有的切割方案都包括进来，所以上述模型并没有精确给出下料问题的等价模型。那么，如何建立出等价的模型呢？一种很直观的方法就是之前提到过的，穷举所有的可行切割方案。根据之前的介绍，这种方法并不可行。那么，我们真的有必要将所有可能的切割方案都囊括进来吗？并不是的。实际上，我们只需要将比较优质的切割方案包括进来就可以。

从单纯形法的迭代过程中可以得知，在目标函数为 min 的 LP 中，若一列的检验数（Reduced Cost）为负值，则该列入基以后，会使得 LP 的目标函数变小，也就是目标函数更优。当所有列中不存在检验数为负的列时，LP 就达到了最优解。因此，对于 min 问题而言，那些检验数为正的列实际上是冗余的，即使它们不参加整个运算过程，LP 同样也可以达到最优解。基于这个想法，我们就把下料问题转化为下面的问题：寻找检验数为负的列，并一一加入模型，直到找不到任何其他检验数为负的列为止。

我们将下料问题模型 (13.3) 称为下料问题的主问题（Master Problem），将寻找新列的问题叫作子问题（Subproblem）或者定价问题（Pricing Problem），或者定价子问题。

但是如何去找这些检验数为负的列，或者说找那些检验数为负的切割方案呢？又或者说如何去构建定价子问题呢？这就要用到单纯形算法中检验数的计算方法了。线性规划中，任给第 j 列对应的决策变量 x_j 的检验数为

$$
(\text{Reduced Cost})_j = \sigma_j = c_j - \boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1} \boldsymbol{N}_j \tag{13.4}
$$

其中，c_j 是变量 x_j 的目标函数系数，$\boldsymbol{c}_B$ 是基变量的目标函数系数向量，而 $\boldsymbol{B}^{-1}$ 则是当前单纯形表对应的可行基中的决策变量对应的初始单纯形表中约束系数矩阵的逆，$\boldsymbol{N}_j$ 是线性规划原问题的约束系数矩阵的第 j 列。我们根据对偶理论得知 $\boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1}$ 就是线性规划原问题中资源的影子价格。也就是对偶变量的取值。因此线性规划原问题的约束对应的对偶变量 $\boldsymbol{y}$（或影子价格）即为

$$
\boldsymbol{y}^{\mathrm{T}} = \boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1} \tag{13.5}
$$

而在本例中，我们是要寻找检验数为负的切割方案，我们用 a_1, a_2, a_3 分别表示一个切割方案中 10 英寸、11 英寸、19 英寸的数量，因此主问题中的一列，就可以表示为 $\boldsymbol{N}_j =$

$(a_1, a_2, a_3)^{\mathrm{T}}$。所以主问题中一列（或者一种切割方案，或者一个决策变量）的检验数就可以表示为

$$\sigma_j = c_j - \boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1} (a_1, a_2, a_3)^{\mathrm{T}} \tag{13.6}$$

但是 a_1, a_2, a_3 的取值必须是整数，且需要满足三者的长度之和不能超过棒材的长度的约束。至此，我们明确了接下来的目标，即不断地找到检验数为负的列，然后加到主问题中，一直循环，直到找不到检验数为负的列为止。具体做法就是，我们可以每次都去寻找检验数最小的切割方案，查看其检验数是否为负。如果检验数最小的切割方案对应的检验数都为正，那说明我们找列的过程就结束了，可以直接去求解最终主问题了。根据上述描述，我们的定价问题就可以表示为

$$\begin{aligned} \min\ z\ &= 1 - \boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1} (a_1, a_2, a_3)^{\mathrm{T}} \\ \text{s.t.}\quad & 10a_1 + 11a_2 + 19a_3 \leqslant 80 \\ & a_1, a_2, a_3 \geqslant 0 \text{ 且为整数} \end{aligned} \tag{13.7}$$

将 $\boldsymbol{c}_B^{\mathrm{T}} \boldsymbol{B}^{-1}$ 替换成对偶变量，即为

$$\begin{aligned} \min\ z\ &= 1 - \boldsymbol{y}^{\mathrm{T}} (a_1, a_2, a_3)^{\mathrm{T}} \\ \text{s.t.}\quad & 10a_1 + 11a_2 + 19a_3 \leqslant 80 \\ & a_1, a_2, a_3 \geqslant 0 \text{ 且为整数} \end{aligned} \tag{13.8}$$

其中，$\boldsymbol{y}$ 是 3×1 的列向量，是主问题 (13.3) 中 3 个约束的对偶变量。这些对偶变量可以通过调用求解器中相应的函数获得。在 Gurobi 中，可以通过 `cons.Pi`（其中 `cons` 是约束类的一个实例）获得；或者通过 `rmp.getAttr("Pi", rmp.getConstrs())` 获得，其中 `rmp` 是一个模型实例。在 CPLEX 中，可以通过 `masterProblem.getDuals(Fill)` 获得，其中 `masterProblem` 是模型实例，`Fill` 是约束实例。

这里还需要提醒读者，检验数为负的列，并不是指将该列加入主问题中，求解主问题之后，得到该新加入的列的检验数为负，而是指假设我们将该新列加入主问题中，基于当前的主问题的最优单纯形表的信息，我们评估该新列的检验数为负。

使用列生成算法求解下料问题，可以通过 Python 调用 Gurobi 比较顺利地完成。具体实现代码见后文。

13.2.2 列生成求解下料问题的一般模型及其伪代码

根据上述描述，我们知道，在列生成算法的过程中，主问题和定价问题/子问题之间是有交互的，交互关系如图 13.2 所示。

上文中是以列生成算法的思路对下料问题进行建模求解的。接下来不用列生成算法的思想，直接对下料问题进行建模，然后再用列生成算法的思想对下料问题的模型进行转化（关于本部分更详细的介绍，见文献（Desaulniers et al.，2006））。

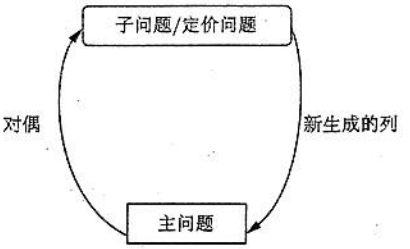


图 13.2 主问题和子问题之间的交互

下料问题的一般描述为：给定长度为 L 的 K 个棒材，以及 $i \in I$ 种长度的棒材需求，每种需求的长度为 s_i，需求量为 d_i，问应该如何切割，使得使用的棒材原料最少？

下面引入决策变量。

（1）$y_k \in \{0,1\}, k \in K$，表示如果第 $k \in K$ 个棒材被使用，则 $y_k = 1$，否则 $y_k = 0$；

（2）$x_{ik} \geqslant 0$ 且为整数，$k \in K, i \in I$，表示切割棒材 k 能够获得的第 i 种需求尺寸材料的数量。

因此，下料问题可以建模为下面的整数规划：

$$z(\mathrm{CSP}) = \min \sum_{k \in K} y_k \tag{13.9}$$

$$\text{s.t.} \sum_{k \in K} x_{ik} \geqslant d_i, \quad \forall i \in I \tag{13.10}$$

$$\sum_{i \in I} s_i x_{ik} \leqslant L y_k, \quad \forall k \in K \tag{13.11}$$

$$\boldsymbol{x} \in \mathbb{Z}_+^n, \quad \boldsymbol{y} \in \{0,1\}^{|K|} \tag{13.12}$$

其中，(13.9) 是最小化使用的棒材数量，约束 (13.10) 表示每种需求必须要被满足；约束 (13.11) 表示如果棒材 k 被切割了，则切割该棒材产生的小棒材的长度之和不超过原棒材的长度。

根据上文的介绍，模型 (13.9) 可以通过列生成的方法，转化为一个主问题和一个子问题，从而将问题化成两个阶段，第 1 个阶段决策出要使用哪些切割方案（产生哪些列）；第 2 个阶段决策出按照每一种切割方案去切割的棒材数量为多少（每一列对应的决策变量的取值是多少）。前者为子问题，后者为主问题。

1. 主问题

首先引入下面的决策变量和参数。引入决策变量 $\lambda_p, \forall p \in P$，表示使用切割方案 p 的棒材的数量。另外，我们引入参数 $a_i^p \geqslant 0$ 且为整数，$\forall i \in I, p \in P$，表示在切割方案 p 中第 i 种尺寸的棒材的数量，该参数可由子问题获得。

准备好上面的参数和变量，我们的主问题就可以写成下面的形式：

$$\min \quad \sum_{p \in P} \lambda_p \tag{13.13}$$

$$\text{s.t.} \quad \sum_{p\in P} a_i^p \lambda_p \geqslant d_i, \qquad \forall i \in I \tag{13.14}$$

$$\lambda_p \geqslant 0 \text{且为整数}, \qquad \forall p \in P \tag{13.15}$$

2. 子问题

接下来就是给出定价子问题。定价子问题的任务就是选出检验数为负的切割方案。首先引入下面的参数和决策变量：

（1）π_i：主问题 (13.13) 中第 i 个约束 (13.14) 的对偶变量（第 i 种需求的数量约束的对偶变量），在定价子问题中，π_i 为已知参数；

（2）$a_i^p \geqslant 0$ 且为整数，$\forall i \in I$：表示在切割方案 p 中第 i 种尺寸的棒材的数量。

注意，这里上标 p 可以省去，因为对于子问题而言，p 实际上是已经被固定了的。

$$\min \quad 1 - \sum_{i\in I} \pi_i a_i^p \tag{13.16}$$

$$\text{s.t.} \quad \sum_{i\in I} s_i a_i^p \leqslant L \tag{13.17}$$

$$a_i^p \geqslant 0 \text{且为整数}, \quad \forall i \in I \tag{13.18}$$

在求解的过程中，我们首先需要初始化主问题，添加一些初始的列。一般来讲，可以用启发式算法生成初始可行列，但是如果简单实现，也可以随意加一些观察就能得到的可行列。完成初始可行列的生成之后，就可以重复上述列生成算法的过程，不断生成检验数为负的列，添加到主问题中，直到没有检验数为负的列为止。最后我们就会得到最终的主问题，求解该主问题即可。

下面我们给出列生成算法的伪代码。

Algorithm 11 列生成算法

```
初始化：生成一系列的初始列 Ω1（例如，用简单的启发式算法）
令 ε ← 非常小的负数容差（例如 −0.001）
求解 MP(Ω1)，得到对偶变量 π
构建 subproblem（pricing problem）SP(π)
得到 subproblem 的目标函数 σ ← 求解 SP(π)
while σ ⩽ ε do
    a ← 根据 subproblem 的解生成新列（一列或多列）
    Ω1 ← Ω1 ∪ a
    π ← 求解 MP(Ω1) 并得到对偶变量
    更新 subproblem SP(π)
    σ ← 求解 SP(π)
end while
求解最终的带整数约束的 MP
```

13.2.3 列生成最优性的几个小问题

通过前文的介绍，我们了解到列生成算法可以通过模型转化，将大规模的问题转化成主问题和子问题（或者若干子问题），而主问题和子问题相较原问题而言，求解起来更容易。因此这种转化可以提高求解效率。但是这里有几点需要注意。一般来讲，主问题实际上是一个整数规划，但是在生成新列的过程中，我们暂时将主问题的整数约束松弛掉，变成一个线性规划，求解该线性松弛问题，并获得主问题所有约束的对偶变量的取值，然后将对偶变量的取值传给子问题。循环上述过程，直到没有新列产生，我们就得到了最终的主问题的形式。在求解最终的主问题时，我们可以将主问题中的所有决策变量设置成整数变量，然后再求解，因为在下料问题中，棒材的切割数量必须是整数。在上述整个过程中，由于我们先将主问题做了松弛处理，最终又把主问题修改成了整数规划，这些操作很可能会导致加入的列会有遗漏，也就是在最优解对应的基（最优基）中的基变量对应的列没有被全部加进来，进而导致得到的解不一定是最优解。导致这个结果的原因是我们用线性规划中变量的检验数去近似评估整数规划的解是否会有改进，这一步近似带来了误差。与前文描述的一致，以下料问题为例，我们用 Ω 表示所有可行的切割方案的集合，用列生成算法得到的列的集合为 Ω'，很明显 $\Omega' \subset \Omega$。如果穷举了所有的可行列，也就是直接得到了完整的 Ω，那么此时形成的主问题一定能保证主问题的解是最优解。如果只是挑出了一些“比较好的”列，最终得到的主问题被称为限制性主问题（Restricted Master Problem，RMP）。

实际上很多时候，RMP 并不一定能等价地代表主问题，因此 RMP 并不能保证最优性。不过，当列生成算法结束时得到的最终 RMP 的线性松弛问题的最优解也是正整数解时，此时 RMP 的最优解同时也是原问题的最优解。从另一个角度理解，就是此时 RMP 的凸包 $\mathbf{Conv}(\Omega')$ 包含了原问题的最优解。当 RMP 的线性松弛最优解是小数，且全局上下界仍存在差距时，说明我们遗漏了一些原问题的最优基中的基变量，也就是说 $\mathbf{Conv}(\Omega')$ 没有包含原问题的最优解。此时我们需要采取其他的办法，来获得最优解。例如，在生成所有检验数为负的列之后，继续用一种被称为定界方法（Bounding method）的办法，将符合界限要求的还未被加入的列添加到 Ω' 中，从而保证最优性，具体的定界方法见文献（Baldacci and Mingozzi，2009）。或者将列生成算法和分支定界算法嵌套在一起使用，变成所谓的分支定价算法（Branch and Price Algorithm）。分支定价算法是能保证求得的解是原问题的最优解的。关于分支定价算法更细节的介绍，我们放在后面的章节。

关于列生成算法的最优性，还有另外一种角度的解释。我们记完备主问题的线性松弛问题为 LMP，记限制性主问题（Restricted Master Problem）的线性松弛问题为 RLMP。注意主问题是和原问题完全等价的，但是限制性主问题不一定和原问题等价。所以主问题的下界一定是原问题的下界，但是限制性主问题的下界不一定是原问题的下界。不同问题，下界的计算方式也不相同。我们以 VRPTW 为例来介绍列生成算法中原问题下界的计算。根据文献（Archetti et al.，2011）中的论述，类似的论述见文献（Desaulniers et al.，2006），LMP 和 RLMP 的最优值和原问题最优值 z^* 满足下面的关系：

$$\underline{z}^* = z^*_{\mathrm{LMP}} \geqslant z^*_{\mathrm{RLMP}} + \sum_{s\in S} \bar{c}^*_s \tag{13.19}$$

其中，z^*_{LMP} 是 LMP 的最优值（注意，LMP 的最优解不一定是整数解，但它提供的界限可以给原问题提供全局最优性证明。），z^*_{RLMP} 是当前迭代步骤中对应的 RLMP 的最优值，$\bar{c}^*_s$ 是第 s 个子问题中找到的最优的列的检验数（也就是第 s 个子问题的目标函数），$\underline{z}^*$ 为全局最优解的下界。而 RMP 的整数最优解，就是全局最优解的上界（同时也是 MP 的上界），即

$$\bar{z}^* = z^*_{\mathrm{RMP}} \tag{13.20}$$

因此，在列生成算法的每步迭代中，最优值 z^* 均满足

$$z^*_{\mathrm{RLMP}} + \sum_{s\in S} \bar{c}^*_s \leqslant z^* \leqslant z^*_{\mathrm{RMP}} \tag{13.21}$$

在列生成算法结束后，最终形式的 RMP 的线性松弛 RLMP 的最优解如果是整数解，即 $z^*_{\mathrm{RLMP}} = z^*_{\mathrm{RMP}}$，且由于列生成算法已经结束了，所以每个子问题的目标值均为非负值（接近于 0），即 $\bar{c}^*_s \approx 0,\ \forall s \in S$。此时全局最优解 z^* 满足 $z^*_{\mathrm{RLMP}} + \sum_{s\in S} \bar{c}^*_s = z^*_{\mathrm{RLMP}} + 0 \leqslant z^* \leqslant z^*_{\mathrm{RMP}}$，进而有 $z^*_{\mathrm{RLMP}} = z^* = z^*_{\mathrm{RMP}}$，即此时 RMP 的整数最优解，同时也是原问题的最优解。如果 RMP 的线性松弛 RLMP 的最优解不是整数解，就一定有 $z^*_{\mathrm{RLMP}} < z^*_{\mathrm{RMP}}$，此时，虽然有 $\bar{c}^*_s \approx 0,\ \forall s \in S$，但是全局最优值 z^* 的最优界限却为 $z^*_{\mathrm{RLMP}} + 0 < z^* \leqslant z^*_{\mathrm{RMP}}$。由于下界和上界之间仍然存在差距（即 Gap > 0），因此此时并不能保证 RMP 的整数解就是原问题的最优解。这种从界限角度的解释，也许更容易理解。

13.3 列生成求解下料问题的实现

本节我们会给出若干版本的下料问题的实现代码，供读者参考。在给出代码前，我们首先来回顾前面部分介绍的例子。

限制性主问题为最小化使用的棒材原料的数量，即

$$\begin{aligned}
\min z = {} & x_1 + x_2 + x_3 + x_4 + x_5 && && && \text{(RMP)}\\
\text{s.t.} \quad & 8x_1+6x_2+6x_3+5x_4+5x_5 && \geqslant && 46\\
& x_2 + x_4+2x_5 && \geqslant && 22\\
& x_3 + x_4 && \geqslant && 43\\
& x_1,\ x_2,\ x_3,\ x_4,\ x_5 && \geqslant && 0 \text{ 且为整数}
\end{aligned}$$

子问题为

$$\begin{aligned}
\min\ z = {} & 1 - \boldsymbol{y}^{\mathrm{T}}(a_1, a_2, a_3)^{\mathrm{T}} && && && \text{(SP)}\\
\text{s.t.} \quad & 10a_1 + 11a_2 + 19a_3 && \leqslant && 80\\
& a_1, a_2, a_3 && \geqslant && 0 \text{ 且为整数} && (13.22)
\end{aligned}$$

其中，$\boldsymbol{y}$ 是 3×1 的列向量，是 RMP 中 3 个约束的对偶变量。

13.3.1 Python 调用 Gurobi 实现列生成求解下料问题示例算例：版本 1

cutstockExampleInstance.py 代码如下：

cutstockExampleInstance.py

```
from gurobipy import *
# Number of different width products
itemNum = 3

# Base width of the rolls
Length = 80

# item width and demand
l = {0: 10, 1: 19, 2: 11}  # width of each item
d = {0: 46, 1: 43, 2: 22}  # demand of each item

# construct the subproblem
SP = Model('subproblem')
SPvars = SP.addVars(itemNum, obj=-1, vtype=GRB.INTEGER, name='w')
        # set objective - a1 - a2 - a3
SP.addConstr(SPvars.prod(l) <= Length) # set objective 10 a1 + 11 a2 + 19
        # a3 <= 80
SP.write('subproblem.lp')
SP.update()

# Construct initial master problem
MP = Model('Master Problem')
x = {}
for i in range(5):
    x[i] = MP.addVar(obj=1, vtype=GRB.CONTINUOUS, name='x_' + str(i))
cons = {}
cons[0] = MP.addConstr(8 * x[0] + 6 * x[1] + 6 * x[2] + 5 * x[3] + 5 * x[4]
       >= 46, name = 'c_1')     # set constraint 1
cons[1] = MP.addConstr(x[1] + x[3] + 2 * x[4] >= 22, name = 'c_2')
        # set constraint 2
cons[2] = MP.addConstr(x[2] + x[3] >= 43, name = 'c_3')
        # set constraint 3
MP.write('initial_MP.lp')
# We have just set-up the initial master problem with initial variables
MP.optimize()

# We will loop as long as we find interesting variables
MP.Params.OutputFlag = 0
```

```
Iter = 0
eps = -0.0001   # tolerance
while(MP.Status == GRB.OPTIMAL):
    pi = { i : -cons[i].Pi for i in range(itemNum)}
    SP.setObjective(SPvars.prod(pi))
    SP.optimize()
    # This should not happen... but better safe than sorry
    if(SP.Status != GRB.OPTIMAL):
        raise('Unexpected optimization status')
    # Reduced cost is c_i - pi * A_{*i}
    # - in our case, A_{*i} is the proposed solution in SP
    # - pi comes from the duals in MP
    # - c_i is the cost in the master problem for the column, which in this
      # case is just # If improvement is too small, just stop
    if(1 + SP.ObjVal > eps):   # notice that the model built above ignore a
         # constant 1
        break
    # Log
    if(Iter % 10 == 0):
        print('Iteration MasterValue PricingValue')
    print('%8d %12.5g %12.5g' % (Iter, MP.ObjVal, SP.ObjVal))
    Iter += 1

    # Using solution, build new variable
    col = Column()
    for j in range(itemNum):
        col.addTerms(SPvars[j].X, cons[j])
    MP.addVar(obj=1, column=col, name = 'new_x' + str(Iter))
    MP.optimize()
MP.write('final_MP.lp')
rootbound = MP.ObjVal

MP.Params.OutputFlag = 1
Mvars = MP.getVars()
for v in Mvars:
    v.VType = GRB.INTEGER   # change the variable's type to INTEGER
MP.optimize()
```

根据上述代码的输出，我们最终的 RMP 存储在 `final_MP.lp` 文件中，其具体信息如下：

final MP.py

```
\ Model Master Problem
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  x_0 + x_1 + x_2 + x_3 + x_4 + new_x1 + new_x2 + new_x3
Subject To
 c_1: 8 x_0 + 6 x_1 + 6 x_2 + 5 x_3 + 5 x_4 >= 46
 c_2: x_1 + x_3 + 2 x_4 + 7 new_x2 + 2 new_x3 >= 22
 c_3: x_2 + x_3 + 4 new_x1 + 3 new_x3 >= 43
Bounds
End
```

我们将所有变量均设置成整数变量，即变成下面的整数规划模型：

$$
\begin{aligned}
\min z = {} & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 && \text{(RMP)}\\
\text{s.t.} \quad & 8x_1+6x_2+6x_3+5x_4+5x_5 \geqslant 46\\
& x_2 + x_4+2x_5 +7x_7+2x_8 \geqslant 22\\
& x_3 + x_4 +4x_6 +3x_8 \geqslant 43\\
& x_1,\ x_2,\ x_3,\ x_4,\ x_5,\ x_6,\ x_7,\ x_8 \geqslant 0 \text{ 且为整数}
\end{aligned}
$$

我们继续调用 Gurobi 求解，运行结果如下：

final MP.py

```
Objective =       20.0
x_0 =     5.0
x_1 =     0.0
x_2 =     2.0
x_3 =     0.0
x_4 =     0.0
new_x1 =          2.0
new_x2 =          0.0
new_x3 =          11.0
```

根据结果，我们得知最终 RMP 的最优解中选择的切割方式有 4 种，如图 13.3 所示。

我们观察到，最终 RMP 中仅有 3 个约束，但是带有整数约束的 RMP 的最优解中有 4 个变量为正。这说明最终 RMP 的线性松弛问题的最优解不是整数解，因此上述 RMP 的解并不一定是原问题的最优解。若要证明该解是否为最优解，需要计算此时的全局下界。

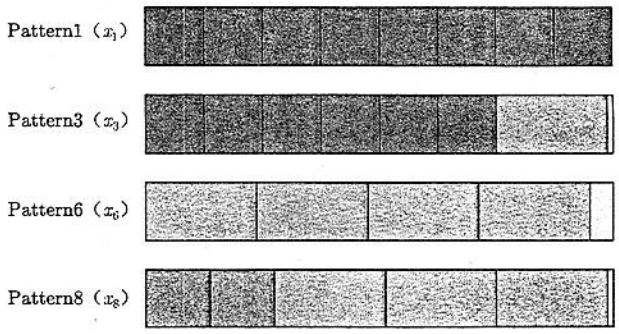


图 13.3 最终被选中的切割方案

13.3.2 Python 调用 Gurobi 实现列生成求解下料问题示例算例：版本 2（以人工变量为初始列的方式）

在上述章节中，我们的做法是，在列生成算法迭代开始前，首先在主问题中添加了一些初始的列，以此来完成主问题的初始化。当然，主问题列的产生也可以通过启发式算法完成，还可以通过添加一些无用的初始列的办法完成（这些列是绝对不会出现在最优解里面的，加入它们的唯一目的就是使得算法能够进行下去）。

本节我们就来展示如何用添加人工变量的方法，为主问题构造初始可行列，初始化主问题，并完成整个列生成算法的实现。

我们将初始的限制性主问题设置成下面的形式：

$$\begin{aligned} \min \quad & z = 1000s_1 + 1000s_2 + 1000s_3 \\ \text{s.t.} \quad & s_1 \geqslant 46 \\ & s_2 \geqslant 22 \\ & s_3 \geqslant 43 \\ & s_1, s_2, s_3 \geqslant 0 \end{aligned} \tag{13.23}$$

其中，变量 s_1, s_2, s_3 其实相当于松弛变量，一定不会出现在最优解中。

我们可以将代码写得更通用一些，自动地生成各种参数的算例，并且求解。具体代码如下。

cutStock Example Instance 2.py

```
from gurobipy import *
from random import randint
from math import floor

# Number of different width products
itemNum=3  # 50
```

```
7
8  # Set Length of the rolls
9  Length=80  # 750
10 # Length or width of each product
11 item_length={i:randint(10,floor(Length/3)) for i in range(itemNum)}
12 # Demand for each product
13 demand={i:randint(5,50) for i in range(itemNum)}   # 5, 200
14
15 # construct the subproblem
16 SP = Model('SubProblem')
17 SPvars = SP.addVars(itemNum, obj=-1, vtype=GRB.INTEGER, name='w')
18 SP.addConstr(SPvars.prod(item_length) <= Length) # set objective 10 a1 + 11
       # a2 + 19 a3 <= 80
19 SP.update()
20
21 # construct the master problem
22 MP = Model('Master Problem')
23 slack = MP.addVars(itemNum, name='slackVar',obj=10000 )
24 cons = MP.addConstrs( slack[i] >= demand[i] for i in range(itemNum))
25 MP.write('initialMP.lp')
26 # Solve the initial master problem
27 MP.optimize()
28
29 # We will loop as long as we find interesting variables
30 MP.Params.OutputFlag = 0
31 Iter = 0
32 eps = -0.0001   # tolerance
33 while(MP.Status == GRB.OPTIMAL):
34     pi = { i : -cons[i].Pi for i in range(itemNum)}
35     SP.setObjective(SPvars.prod(pi))
36     SP.optimize()
37     # This should not happen... but better safe than sorry
38     if(SP.Status != GRB.OPTIMAL):
39         raise('Unexpected optimization status')
40     # Reduced cost is c_i - pi * N_{*i}
41     # - N_{*i} is the cutting pattern corrsponding to subproblem
42     # - pi comes from the duals in MP
43     if(1 + SP.ObjVal > eps):   # notice that the model built above ignore a
         # constant 1
44         break
45     # print Log each 10 iterations
46     if(Iter % 10 == 0):
```

```
        print('Iteration MasterValue PricingValue')
    print('%8d %12.5g %12.5g' % (Iter, MP.ObjVal, SP.ObjVal))
    Iter += 1

    # Using solution, build new variable
    col = Column()
    for j in range(itemNum):
        col.addTerms(SPvars[j].X, cons[j])
    MP.addVar(obj=1, column=col, name = 'new_x' + str(Iter))
    MP.optimize()
MP.write('final_MP.lp')
RelaxedMP = MP.ObjVal

MP.Params.OutputFlag = 1
Mvars = MP.getVars()
for v in Mvars:
    v.VType = GRB.INTEGER    # change the variable's type to INTEGER
MP.optimize()
print('Objective = \t', MP.ObjVal)
for var in MP.getVars():
    print(var.varName, '= \t', var.x)
```

13.3.3 Python 调用 Gurobi 实现列生成求解下料问题：版本 3

本节是第 3 个版本的 Python 调用 Gurobi 实现列生成算法求解下料问题的代码。该代码补充了若干功能性函数，如下。

(1) reportRMP(model)：打印 RMP 的目标函数、变量取值和约束的对偶变量；

(2) reportSUB(model)：打印 Subproblem 的目标函数和变量取值；

(3) reportMIP(model)：打印 MIP 的目标函数和变量取值。

代码其余部分与前两个版本大同小异，主要是给读者提供参考。

cutstock.py 代码如下：

cutstock.py

```
from __future__ import division, print_function

from gurobipy import *


rollwidth = 115
size = [25, 40, 50, 55, 70]
amount = [50, 36, 24, 8, 30]
```

```
nwidth = 5

MAX_CGTIME = 1000

def reportRMP(model):
    if model.status == GRB.OPTIMAL:
        print("Using ", model.objVal, " rolls\n")

        var = model.getVars()
        for i in range(model.numVars):
            print(var[i].varName, " = ", var[i].x)
        print("\n")

        con = model.getConstrs()
        for i in range(model.numConstrs):
            print(con[i].constrName, " = ", con[i].pi)
        print("\n")


def reportSUB(model):
    if model.status == GRB.OPTIMAL:
        print("Pi: ", model.objVal, "\n")

        if model.objVal <= 1e-6:
            var = model.getVars()

            for i in range(model.numVars):
                print(var[i].varName, " = ", var[i].x)
            print("\n")

def reportMIP(model):
    if model.status == GRB.OPTIMAL:
        print("Best MIP Solution: ", model.objVal, " rolls\n")

        var = model.getVars()
        for i in range(model.numVars):
            print(var[i].varName, " = ", var[i].x)

try:
    rmp = Model("rmp")
    sub = Model("sub")

```

```
51    rmp.setParam("OutputFlag", 0)
52    sub.setParam("OutputFlag", 0)
53
54    # construct RMP
55    rmp_var = []
56    for i in range(nwidth):
57        rmp_var.append(rmp.addVar(0.0, GRB.INFINITY, 1.0, GRB.CONTINUOUS, "
          rmp_" + str(i)))
58
59    rmp_con = []
60    row_coeff = [0.0] * nwidth
61    for i in range(nwidth):
62        row_coeff[i] = int(rollwidth / size[i])
63        rmp_con.append(rmp.addConstr(quicksum(rmp_var[j] * row_coeff[j] for
          j in range(nwidth)) >= amount[i], "rmpcon_" + str(i)))
64        row_coeff[i] = 0.0
65
66    rmp.setAttr("ModelSense", GRB.MINIMIZE)
67    # end RMP
68
69    # construct SUB
70    sub_var = []
71    for i in range(nwidth):
72        sub_var.append(sub.addVar(0.0, GRB.INFINITY, 0.0, GRB.INTEGER, "sub_
          " + str(i)))
73
74    sub.addConstr(quicksum(sub_var[i] * size[i] for i in range(nwidth)) <=
          rollwidth, "subcon")
75    # end SUB
76
77    print("                *** Column Generation Loop ***                \n")
78    for i in range(MAX_CGTIME):
79        print("Iteration: ", i, "\n")
80
81        rmp.optimize()
82        reportRMP(rmp)
83
84        rmp_pi = rmp.getAttr("Pi", rmp.getConstrs())
85
86        sub.setObjective(1 - quicksum(sub_var[i] * rmp_pi[i] for i in range(
          nwidth)), GRB.MINIMIZE)
87        sub.optimize()
```

```
        reportSUB(sub)

        if sub.objVal > -1e-6:
            break

        rmp_coeff = sub.getAttr("X", sub.getVars())
        rmp_col = Column(rmp_coeff, rmp_con)
        rmp.addVar(0.0, GRB.INFINITY, 1.0, GRB.CONTINUOUS, "cg_" + str(i),
    rmp_col)

    print("                *** End Loop ***                \n")

    mip_var = rmp.getVars()
    for i in range(rmp.numVars):
        mip_var[i].setAttr("VType", GRB.INTEGER)

    rmp.optimize()
    reportMIP(rmp)

except GurobiError as e:
    print('Error code ' + str(e.errno) + ": " + str(e))

except AttributeError:
    print('Encountered an attribute error')
```

13.4 列生成求解 TSP

13.4.1 TSP 的 1-tree 建模及列生成求解

之前的章节探讨了如何从 1-tree 的角度对 TSP 建模。用 1-tree 建模的前提是：TSP 必须是对称 TSP（Symmetric TSP，STSP），边 (i,j) 和边 (j,i) 的成本（或距离）是一样的。

首先从图论的角度来描述 TSP，然后利用图论为 TSP 重新建模。我们知道，TSP 的可行解是一条闭环，满足图中所有节点的度（degree）均为 2。并且加入消除子环路的约束，模型如下：

$$\min \sum_{e \in E} c_e x_e \tag{13.24}$$

$$\text{s.t.} \sum_{e \in E(i)} x_e = 2, \qquad \forall i \in V \tag{13.25}$$

$$\sum_{e\in E(S)} x_e \leqslant |S|-1, \quad 2\leqslant |S|\leqslant n-1, S\subset V \tag{13.26}$$

$$x_e \in \{0,1\}, \quad \forall e\in E \tag{13.27}$$

其中，$E(i)$ 表示经过点 i 的边的集合；$E(S)$ 表示起点和终点都在点集 S 中的边的集合。

上面的模型可以再加强一些，把第一组 $|V|$ 个约束 (13.25)，即

$$\sum_{e\in E(i)} x_e = 2, \forall i\in V$$

全部加起来，然后除以 2，得到

$$\sum_{e\in E} x_e = |V|$$

这个约束可以加强原来的模型。加强后的模型变为

$$\min \sum_{e\in E} c_e x_e \tag{13.28}$$

$$\text{s.t.} \sum_{e\in E(i)} x_e = 2, \quad \forall i\in V \tag{13.29}$$

$$\sum_{e\in E(S)} x_e \leqslant |S|-1, \quad 2\leqslant |S|\leqslant n-1, S\subset V \tag{13.30}$$

$$\sum_{e\in E} x_e = |V| \tag{13.31}$$

$$x_e \in \{0,1\}, \quad \forall e\in E \tag{13.32}$$

根据前面章节介绍的 1-tree 的定义，TSP 的解是一个特殊的 1-tree。而我们可以穷举所有的 1-tree，然后挑选出成本最小的，且是 TSP 解的 1-tree 即可。因此 TSP 问题可以通过列生成算法求解，而且对应的主问题为一个集分割问题（Set Partitioning Problem）。其 Subproblem 即为寻找高质量的 1-tree。

13.4.2 主问题

根据以上描述，RMP 可以看作已经有一系列的候选 1-tree 的模型，需要从中选出一个成本最小的且符合 TSP 解的 1-tree。假设候选的 1-tree 的集合为 T，则主问题可以写成

$$\min \sum_{t\in T}\left(\sum_{e\in E} c_e x_e^t\right)\lambda^t \tag{13.33}$$

$$\text{s.t.} \sum_{t\in T}\left(\sum_{e\in E(i)} x_e^t\right)\lambda^t = 2, \quad \forall i\in V \tag{13.34}$$

$$\sum_{t\in T}\lambda^t = 1, \tag{13.35}$$

$$\lambda^t \geqslant 0, \qquad \forall t \in T \tag{13.36}$$

在上面模型中，$\sum_{e \in E} c_e x_e^t$ 其实就是第 $t(t \in T)$ 个 1-tree 中的所有边的总成本 c_t，即

$$c_t = \sum_{e \in E} c_e x_e^t$$

并且，$\sum_{e \in E(i)} x_e^t$ 就表示点 i 在第 $t(t \in T)$ 个 1-tree 中的度（degree），用 d_i^t 表示，即

$$d_i^t = \sum_{e \in E(i)} x_e^t$$

经过整理，RMP 可以写成

$$\min \sum_{t \in T} c_t \lambda^t \tag{13.37}$$

$$\text{s.t.} \sum_{t \in T} d_i^t \lambda^t = 2, \quad \forall i \in V \tag{13.38}$$

$$\sum_{t \in T} \lambda^t = 1, \tag{13.39}$$

$$\lambda^t \geqslant 0, \qquad \forall t \in T \tag{13.40}$$

这里总结一下，RMP 就是从一个候选的 1-tree 的集合中，选取出 1 条能够符合 TSP 解的 1-tree，并使得成本最小。

13.4.3 子问题

子问题就是生成一条检验数为负的 1-tree。首先我们需要建立一个生成 1-tree 的数学模型。我们根据 1-tree 的如下图论特性构建数学模型：

（1）有且仅有一个环；

（2）图中每个点的度（degree）$\geqslant 1$；

（3）一共有 $|V|$ 条边。

模型如下所示：

$$\min \sum_{e \in E} c_e x_e \tag{13.41}$$

$$\text{s.t.} \sum_{e \in \delta(1)} x_e = 2 \tag{13.42}$$

$$\sum_{e \in E(S)} x_e \leqslant |S| - 1, \quad \varnothing \subset S \subseteq V \setminus \{1\} \tag{13.43}$$

$$\sum_{e \in E} x_e = n \tag{13.44}$$

$$x_e \in \{0,1\}, \qquad \forall e \in E \tag{13.45}$$

其中，$\delta(1)$ 表示经过点 1 的边的集合，我们需要将其变成子问题/定价问题。为此，我们需要利用主问题的对偶变量来更新子问题的目标函数。

首先，主问题可以简写为

$$\min \ c^{\mathrm{T}}\lambda \tag{13.46}$$

$$\text{s.t. } A\lambda = b \qquad \rightarrow \pi \tag{13.47}$$

$$\lambda \geqslant 0 \tag{13.48}$$

其中，λ 是一个列向量，其中每一个元素 λ_i，$\forall i \in T$ 表示第 i 个 1-tree 对应的决策变量，表示该 1-tree 是否出现在最优解中。理论上来说它应当是一个 0-1 变量，但是这里我们做了松弛，是为了能够让主问题变成线性规划，从而可以得到约束的对偶变量。π 为约束 $A\lambda = b$ 的对偶变量。每一列的检验数为

$$\text{reduced cost}_j = c(x) - \pi^{\mathrm{T}} a(x) \tag{13.49}$$

其中，参数如下：

（1）x 表示一个 1-tree，也就是其中有哪些边，由子问题求解得来；

（2）$c(x) = \sum_{e \in E} c_e x_e$ 表示 1-tree $i \in T$ 的总成本或总长度；

（3）$a(x)$：1-tree $i \in T$ 中每个节点的度（degree）组成的列向量，且有

$$a(x) = \begin{pmatrix} \sum\limits_{i=1 \text{ or } j=1} x_{ij} \\ \sum\limits_{i=2 \text{ or } j=2} x_{ij} \\ \vdots \\ \sum\limits_{i=n \text{ or } j=n} x_{ij} \end{pmatrix}$$

其中，第 $i(i \in V)$ 行代表点 i 的度（degree）。

因此，子问题（SP）可以写为

$$\min \ c(x) - \pi^{\mathrm{T}} a(x) \tag{13.50}$$

$$\text{s.t.} \quad x \in X \tag{13.51}$$

其中，$x \in X$ 表示 x 是一个 1-tree。X 表示所有 1-tree 的集合。

根据上面的式子，我们将检验数进一步改写为

$$c(x) - \pi^{\mathrm{T}} a(x) \tag{13.52}$$

$$=\sum_{e\in E}c_e x_e-\left(\sum_{i\in V}\pi_i\left[\sum_{j\in V}x_{ij}+\sum_{j\in V}x_{ji}\right]\right) \tag{13.53}$$

$$=\sum_{i\in V}\sum_{i\in V}c_{ij}x_{ij}-\left(\sum_{i\in V}\pi_i\left[\sum_{j\in V}x_{ij}+\sum_{j\in V}x_{ji}\right]\right) \tag{13.54}$$

$$=\sum_{i\in V}\sum_{j\in V}x_{ij}\left(c_{ij}-\pi_i-\pi_j\right) \tag{13.55}$$

因此，子问题的最终形式为

$$\min\sum_{i\in V}\sum_{j\in V}x_{ij}\left(c_{ij}-\pi_i-\pi_j\right) \tag{13.56}$$

$$\text{s.t.}\ \sum_{e\in\delta(1)}x_e=2 \tag{13.57}$$

$$\sum_{e\in E(S)}x_e\leqslant|S|-1,\qquad \varnothing\subset S\subseteq V\setminus\{1\} \tag{13.58}$$

$$\sum_{e\in E}x_e=n \tag{13.59}$$

$$x_e\in\{0,1\},\qquad \forall e\in E \tag{13.60}$$

至此，我们完成了主问题和子问题的推导，接下来就可以尝试编程实现了，完整代码见附配资源。

13.5 列生成求解 VRPTW

13.4 节我们介绍了列生成算法求解 TSP，本节将介绍列生成算法求解 VRPTW 的基本原理及其 Java 和 Python 实现。

首先来介绍列生成算法求解 VRPTW 的基本原理（参考自文献 Desaulniers et al., 2006）。

我们回顾之前章节的内容，VRPTW 的解是一系列车辆的路径，每一辆车都对应唯一的一条路径，一条路径实际上就是所有客户的一个子集中所有客户的访问序列。比如某辆车需要访问的客户点的序号集合为 $\{3,7,2\}$，其最优访问顺序为 $0\longrightarrow 3\longrightarrow 7\longrightarrow 2\longrightarrow 0$，这表示该辆车从起始点 0（或者仓库 depot）出发，依次经过客户点 3、7、2，最终返回起始点。这样一条路径，在图上对应一个环（cycle）。也就是一辆车的路径就是一个环。由于车与车之间没有重复访问的客户，因此在可行解中，环与环之间除了起始点，没有其他共同的点。由于一辆车的可行解是非常多的，因此，VRPTW 又可以描述为从一系列的可行环中，为每一辆车选出唯一与之匹配的环，并且这些环满足以下几个条件：

（1）环与环之间除了起始点，没有其他共同点；

（2）所有环恰好覆盖了所有的客户点一次。

基于以上描述，我们可以将求解 VRPTW 问题的过程转化为下面 2 个步骤。第 1 步，为每一辆车找出所有可行的环。第 2 步，从这些可行的环中为每一辆车分配一个环，且满足上面 2 个条件。

那么如何为每一辆车寻找所有的可行环呢？

假设所有的车均是同质的，也就是容量和其他参数都是一致的，容易得出，此时所有车的可行环集合都是相同的。我们只需要找出一辆车的所有可行环即可。假设一辆车的所有可行环的集合为 Ω，为了列举出 Ω 中的所有元素，我们可以通过穷举，找出 Ω 中的所有可行环。但是这些可行环的集合的元素个数可能是随着算例的增大呈指数级增长的，要想将它们全部找到，很多时候并不可行。我们不妨换一种思路，不去暴力枚举所有的可行环，而只是尝试去找那些比较优质的环。跟之前介绍的下料问题和 TSP 的列生成算法求解的思路一致，我们只选择那些检验数为负的可行环即可。我们用 Ω' 表示这些检验数为负的可行环的集合。显然，$\Omega' \subset \Omega$。

确定了集合 Ω' 之后，我们就可以为每一辆车安排一个环且使得总的行驶距离最小。该问题可被建模成为一个集分割问题，这也正是 VRPTW 对应的 RMP。

而找到这些检验数为负的可行环集合的问题，就是子问题。该子问题可以转化成一个带资源约束的基本最短路问题（Elementary Shortest Path Problem with Resource Constraints，ESPPRC）。即，一辆车从起始点出发，访问客户点中的一部分（至多访问一次），并且满足这些客户点的需求和时间窗约束，最终回到起始点。

下面给出主问题和子问题的数学模型。注意，本章忽略了车辆数限制，在实际问题中，读者可将其加入到模型中。

13.5.1 主问题

用 P^k 表示车辆 $k \in V$ 的所有可行环，则对于 P^k 中任意一条 cycle p 都对应一组 0-1 变量 x_{ijp}^k，当 $x_{ijp}^k = 1$ 表示车辆 k 在 cycle p 上从 i 点到 j 点；当 $x_{ijp}^k = 0$ 表示车辆 k 不经过 cycle p 上的弧 (i,j)。因此，我们可以得到下面的等式：

$$x_{ij}^k = \sum_{p \in P^k} x_{ijp}^k y_p^k, \qquad \forall k \in V, \forall (i,j) \in A \tag{13.61}$$

$$\sum_{p \in P^k} y_p^k = 1, \qquad \forall k \in V \tag{13.62}$$

$$y_p^k \geqslant 0, \qquad \forall k \in V, \forall p \in P^k \tag{13.63}$$

其中，x_{ij}^k 为 VRPTW 中的路径决策 0-1 变量，表示车辆 k 是否经过弧 (i,j)。这里，y_p^k 表示第 k 辆车是否选择经过 cycle p。我们也可以定义出 cycle p 的成本 c_p^k 和顾客 i 在路径 p 被车辆 k 拜访的次数 $a_{ip}^k \in \{0,1\}$，即

$$c_p^k = \sum_{(i,j) \in A} c_{ij}^k x_{ijp}^k, \qquad \forall k \in V, \forall p \in P^k \tag{13.64}$$

$$a_{ip}^k = \sum_{j \in N} x_{ijp}^k, \qquad \forall k \in V, \forall i \in C, \forall p \in P^k \tag{13.65}$$

下面用这些信息来给出 VRPTW 的主问题的模型如下：

$$\min \quad \sum_{k \in V} \sum_{p \in P^k} c_p^k y_p^k \tag{13.66}$$

$$\text{s.t.} \quad \sum_{k \in V} \sum_{p \in P^k} a_{ip}^k y_p^k = 1, \qquad \forall i \in C \tag{13.67}$$

$$\sum_{p \in P^k} y_p^k = 1, \qquad \forall k \in V \tag{13.68}$$

$$y_p^k \geqslant 0, \qquad \forall k \in V, \forall p \in P^k \tag{13.69}$$

由于考虑所有车都是同质的，所以所有的 P^k 都是相同的，也就是说 $P^k = P$。不难得出，所有的子问题也都是等价的。因此，约束 (13.68) 可以消除下标 k，从而被简化。于是我们得到了 VRPTW 的主问题的模型，该模型是一个集分割问题模型。这里直接给出主问题的线性松弛形式，即

$$\min \quad \sum_{k \in V} \sum_{p \in P} c_p y_p \tag{13.70}$$

$$\text{s.t.} \quad \sum_{p \in P} a_{ip} y_p = 1, \qquad \forall i \in C \tag{13.71}$$

$$y_p \geqslant 0, \qquad \forall p \in P \tag{13.72}$$

与前文类似，考虑在列生成算法中，主问题中的列的集合是被限制成了已经被生成的列。下面用 $P' \subset P$ 表示列生成算法产生的 cycle 的集合，则 VRPTW 的主问题可以写成 RMP 的形式，即

$$\min \quad \sum_{k \in V} \sum_{p \in P} c_p y_p \tag{13.73}$$

$$\text{s.t.} \quad \sum_{p \in P} a_{ip} y_p = 1, \qquad \forall i \in C \tag{13.74}$$

$$y_p \geqslant 0, \qquad \forall p \in P' \tag{13.75}$$

其中，每个决策变量 y_p 表示路径 p 被使用的次数。这个决策变量不一定是整数，但是是非负的任意实数。a_{ip} 表示客户 i 在路径 p 上被服务的次数，并且 c_p 表示 cycle p 的成本。参数 a_{ip} 原则上应该取 0 或者 1，但是由于子问题可以是松弛后的问题，所以它可以取大一点的整数。这里有一点需要特别说明，上述 RMP 中，约束 (13.75) 做了一步松弛，本来应该是 $0 \leqslant y_p \leqslant 1$，但是我们将其松弛成了 $y_p \geqslant 0$。这一步松弛是等价的转化，因为在上述 RMP 模型的最优解中，y_p 的取值不可能大于 1，换句话说，如果 RMP 的一个解对应的 y_p 取值大于 1，则该解一定不是 RMP 的最优解。

13.5.2 子问题

VRPTW 的每个子问题，就是获得一条具有负检验数的 cycle，根据前文介绍，该问题是一个 ESPPRC。即，每一辆车只访问所有客户点中的一部分点，并且在满足载重、时间窗等约束的情况下，使得目标函数最小。子问题的目标函数就是新列（新 cycle）的检验数，即

$$\begin{aligned}\text{obj} = \min \quad & c_p - \sum_{i \in C} \lambda_i a_i \\ &= \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij} - \sum_{i \in C} \lambda_i a_i \\ &= \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij} - \sum_{i \in C} \lambda_i \sum_{j \in N} x_{ij} \\ &= \sum_{i \in N} \sum_{j \in N} (c_{ij} - \lambda_i) x_{ij} \end{aligned} \tag{13.76}$$

其中，a_i 就是 RMP 中的 a_{ip}。因为子问题就是得到一条满足资源约束的最短路，而不必强调车辆的编号 k，因此我们可以去掉下标 k。此时 $a_{ip}^k = a_{ip} = \sum_{j \in N} x_{ij} = 0$ or 1，a_{ip} 也正是添加到主问题中的列的约束系数。此外，λ_i 是 RMP 的第 i 条约束的对偶变量。

因此，ESPPRC 的模型如下：

$$\min \quad \sum_{i \in N} \sum_{j \in N} (c_{ij} - \lambda_i) x_{ij} \tag{13.77}$$

$$\sum_{i \in C} d_i \sum_{j \in N} x_{ij} \leqslant q, \tag{13.78}$$

$$\sum_{j \in N} x_{0j} = 1, \tag{13.79}$$

$$\sum_{i \in N} x_{ih} - \sum_{j \in N} x_{hj} = 0, \qquad \forall h \in C \tag{13.80}$$

$$\sum_{i \in N} x_{i,n+1} = 1, \tag{13.81}$$

$$s_i + t_{ij} - M(1 - x_{ij}) \leqslant s_j, \qquad \forall i, j \in N, i \neq j \tag{13.82}$$

$$a_i \leqslant s_i \leqslant b_i, \qquad \forall i \in N \tag{13.83}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i, j \in N, i \neq j \tag{13.84}$$

可以看到，子问题的约束中没有了 $\forall k \in K$ 这个要求，因此从规模上来讲，ESPPRC 比 VRPTW 缩小了 K 倍，也更容易求解。但是 ESPPRC 仍旧是 NP-hard 问题，虽然相比 VRPTW 更容易，但是一旦规模加大，也并不能保证很快得到最优解。幸运的是，ESPPRC 可以使用动态规划的方法高效地求解，具体来说就是标签算法（Labeling Algorithm）。求解 ESPPRC 的标签算法实际上是一种标签校正算法，对应地，还有一种被称为标签设定算法

的算法，比如 Dijkstra 算法。标签算法本身是一种穷举的思想，但是由于加入了优超准则（Dominance Rule），使得它在求解特定问题时有较高的效率。正因为如此，如果 ESPPRC 使用标签算法去求解（而不是直接调用求解器求解），那么我们可以求解规模稍微大一些的 VRPTW 算例。但是在本章，暂时不涉及标签算法的部分。本章的代码中，子问题的求解全部直接调用求解器求解，旨在展示列生成算法的过程。关于标签算法的详细介绍，以及用列生成算法结合标签算法求解 VRPTW 的代码，请参照动态规划部分的内容。

13.5.3 详细案例演示

为了让读者更直观地理解列生成算法求解 VRPTW 的完整流程，我们以 Solomon VRP Benchamrk 算例中的 C101 为例，使用 Python 调用 Gurobi 来逐步展示整个算法迭代过程。为了方便展示，我们只提取前 5 个顾客点为输入数据，并且设置车辆数为 5。

首先，我们需要构造初始 RMP，即添加初始可行列。初始 RMP 的构造一定要保证问题有可行解。为了简化处理，我们让每辆车只经过一个顾客点，即为每个顾客分配一辆车（这种构造初始可行列的方法非常低效，在实际科研中，我们一般用其他方法构造初始可行列，比如使用启发式算法生成初始可行列）。RMP 模型的行数等于顾客的数量，即 5 行，对应 5 个约束，每个约束表示对应顾客一定要被访问一次。按照上述方法操作完成后，初始 RMP 可以写成下面的形式。

$$
\begin{aligned}
&\min \quad c_1y_1+c_2y_2+c_3y_3+c_5y_4+c_5y_5 && (13.85)\\
&\text{node 1} \quad y_1 && =1\\
&\text{node 2} \quad y_2 && =1\\
&\text{node 3} \quad y_3 && =1\\
&\text{node 4} \quad y_4 && =1\\
&\text{node 5} \quad y_5 && =1\\
&y_1,\ y_2,\ y_3,\ y_4,\ y_5 && \in\{0,1\}
\end{aligned}
$$

其中，第 1 列对应路径 1，为（0 − 1 − 0），其路径长度为 c_1。第 2 列对应路径 2，为（0 − 2 − 0），其路径长度为 c_2。其他列类似。

我们将上述 RMP 松弛成线性规划，然后使用 Python 调用 Gurobi 对其求解。为了方便实现，我们添加虚拟点 6，作为 depot 0 的副本，即点 1 到点 5 为顾客点，点 0 和点 6 为同一个点，都代表 depot。相应地，路径（0−1−0），（0−2−0）就变成了（0−1−6），（0−2−6）。为了节省篇幅，我们省略读入数据的部分，仅展示算法核心部分的代码。

首先，我们初始化 RMP，代码如下：

VRPTW Column Generation small instance

```
RMP = Model('RMP')
customerNum = data.customerNum
```